

Article

From Hand-Crafted Features to Large Language Models: A Comparative Evaluation of Android Malware Detection Paradigms

Egemen Taşkın ^{1,*}  and İbrahim Alper Doğru ² ¹ Department of Information Security Engineering, Graduate School of Natural and Applied Sciences, Gazi University, Ankara 06560, Turkey² Department of Computer Engineering, Faculty of Technology, Gazi University, Ankara 06560, Turkey; iadogru@gazi.edu.tr

* Correspondence: egementaskin@gmail.com

Abstract

The rapid evolution of Android malware and increasingly sophisticated obfuscation techniques challenge traditional detection systems. This study presents a rigorous, unified comparative evaluation of three methodological paradigms—classical machine learning, Transformer-based architectures, and generative Large Language Models (LLMs)—for static Android malware detection. We construct a balanced dataset of 12,000 APKs from the AndroZoo repository and implement a fold-independent experimental pipeline featuring constraint-aware sequence selection for Transformers and structured LLM-driven feature distillation with parameter-efficient fine-tuning (LoRA). All evaluations employ stratified 5-fold cross-validation with statistical significance testing and comprehensive resource profiling. Classical models (e.g., Random Forest) achieve strong baselines (~0.975 F1) but exhibit limited contextual resilience. Distilled Transformers (RoBERTa ~0.970 F1-score) deliver an optimal accuracy-latency trade-off for real-time screening. While zero-shot LLMs show moderate performance (~0.74–0.84 F1), integrating LLM-extracted semantic features with LoRA fine-tuning yields accuracy (Qwen3.5-27B: ~0.982 F1-score), cross-dataset generalization, and structured interpretability. Hallucination analysis reveals a manageable 7.7% rate, with ablation confirming minimal impact on downstream classification. We advocate a tiered deployment strategy: lightweight Transformers for high-throughput screening, complemented by fine-tuned LLMs for deep forensic analysis and explainable threat intelligence. This hybrid framework effectively balances computational efficiency, detection robustness, and operational interpretability for modern Android security pipelines.

Keywords: Android malware detection; static analysis; transformer models; large language models; LLM-based feature extraction



Academic Editor: Douglas
O'Shaughnessy

Received: 7 May 2026
Revised: 31 May 2026
Accepted: 2 June 2026
Published: 3 June 2026

Copyright: © 2026 by the authors.
Licensee MDPI, Basel, Switzerland.
This article is an open access article
distributed under the terms and
conditions of the [Creative Commons
Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

The Android operating system has emerged as the dominant mobile platform globally, commanding a significant share of the smartphone market. This widespread adoption, coupled with the openness of its application ecosystem, has made Android a prime target for malicious actors. The continuous proliferation of Android malware poses severe threats to user privacy, financial security, and device integrity, necessitating robust, scalable, and adaptive detection mechanisms. Over the past decade, research in this domain has undergone a profound methodological evolution, transitioning from rule-based heuristics

and classical machine learning to sophisticated deep learning architectures and, most recently, generative Large Language Models (LLMs). Each paradigm has sought to address the limitations of its predecessors, particularly in the face of increasingly advanced code obfuscation and evasion techniques.

Despite this rapid progression, a comprehensive, empirically grounded comparative evaluation spanning classical machine learning, Transformer-based architectures, and LLM-driven frameworks remains notably absent. Existing studies typically isolate a single generation of techniques, rely on heterogeneous datasets, or employ inconsistent evaluation protocols, obscuring the practical trade-offs between detection accuracy, computational overhead, and interpretability. To bridge this gap, this study conducts a systematic comparative evaluation across all three paradigms using a unified experimental pipeline. We construct a balanced dataset of 12,000 applications from the AndroZoo repository and implement standardized protocols encompassing classical classifiers (e.g., Random Forest, SVM), fine-tuned Transformer models (BERT, RoBERTa, DistilBERT), and LLM-based approaches (zero-shot prompting, LLM-assisted feature extraction, and parameter-efficient fine-tuning via LoRA). By unifying feature engineering, sequence handling, and validation metrics, we provide practitioners with actionable insights into the operational deployment of each paradigm.

The principal contributions of this work are threefold:

1. **Unified Comparative Framework:** We establish a reproducible, end-to-end evaluation pipeline that enables direct, apples-to-apples comparison across classical, Transformer, and LLM-based detection paradigms under consistent data preprocessing and validation conditions.
2. **Constraint-Aware Sequence Modeling:** We introduce a novel hybrid feature engineering and constraint-aware sequence selection methodology that optimizes the adaptability of Transformer architectures to high-dimensional, heterogeneous static analysis data while adhering to token-length limitations.
3. **LLM-Driven Feature Distillation Pipeline:** We design and validate a structured prompt-engineering framework for extracting semantically enriched security indicators via LLMs, demonstrating how these distilled features can be effectively integrated with parameter-efficient fine-tuning (LoRA) to enhance downstream classifier performance.

While individual paradigms have advanced rapidly, the current literature remains fragmented. Existing studies typically evaluate these approaches in isolation, rely on heterogeneous preprocessing pipelines, or omit critical deployment constraints such as inference latency, memory footprint, and cross-dataset generalization. Moreover, while LLMs show promise for interpretability, their practical integration into security workflows is hindered by unquantified hallucination rates, ad hoc prompt designs, and prohibitive computational costs. This study addresses these gaps by introducing the first end-to-end, fold-independent comparative framework that standardizes feature extraction, sequence handling, and validation protocols across all three paradigms. Crucially, we propose a constraint-aware sequence selection mechanism that resolves Transformer token-length bottlenecks without arbitrary truncation, and a structured LLM-driven feature distillation pipeline coupled with LoRA fine-tuning that transforms generative reasoning into parameter-efficient, reproducible classifiers. By rigorously quantifying resource consumption, hallucination impact, and cross-dataset transferability, this work moves beyond isolated accuracy claims to provide practically oriented insights that may inform the design of modern Android security pipelines.

The remainder of this paper is organized as follows: Section 2 reviews related work and systematically analyzes the performance trade-offs across methodological generations.

Section 3 details the dataset construction, feature engineering, and experimental methodology. Section 4 presents the comparative evaluation results across classical, Transformer-based, and LLM-driven paradigms. Finally, Section 5 concludes the study and outlines directions for future research.

2. Related Work

2.1. Classical Static Analysis and Hand-Crafted Features

The foundation of modern Android malware detection rests upon the principle of static analysis, a technique that examines an application without executing it [1,2]. This approach stands in contrast to dynamic analysis, which requires running the application in a controlled sandbox environment to observe its behavior [1]. The primary advantage of static analysis is its efficiency and scalability; it can process vast repositories of applications, such as those found in public app stores, without the time-consuming and potentially hazardous task of execution [3,4]. By analyzing an application package (APK), developers and security researchers can extract a rich set of metadata and code-level information, forming the basis for automated detection systems [5,6]. The most widely adopted features in this domain include permissions requested by the application, the sequence of Application Programming Interface (API) calls made during its operation, and the interactions defined through intents [6–8]. These features serve as a proxy for an application's intended functionality and potential malicious intent [8].

Permissions represent one of the most fundamental categories of static features. They are declared within the `AndroidManifest.xml` file and grant an application access to protected hardware, user data, and other resources [9,10]. For instance, an application requesting permission to read SMS messages or make phone calls might be flagged for further inspection [11]. The seminal work of the Drebin project established the use of permissions as a core component of its lightweight detection methodology [10,12]. Drebin collected a diverse set of features, including permissions, intents, application components, API calls, and network addresses, to build a comprehensive profile of each application [10]. However, the utility of permissions as a standalone feature is limited. Malware authors can easily circumvent detection by either over-requesting benign permissions to blend in with legitimate apps or by using fine-grained permissions that appear less suspicious [13,14]. Furthermore, some permissions themselves may not directly imply maliciousness but are required for legitimate functionality, leading to ambiguity for any rule-based system [15]. Despite these limitations, permissions remain a popular feature due to their ease of extraction and direct link to user-facing privileges [11].

A more granular and powerful set of features comes from API calls. Unlike permissions, which only state an intention, API calls reveal the actual operations performed by the application at runtime [16]. These calls are extracted from the Dalvik Executable (DEX) bytecode, which contains the application's compiled code [17]. Tools like Androguard provide the necessary infrastructure to parse DEX files, allowing analysts to build graphs of methods, identify callgraphs, and locate specific API usages [18]. The sequence of API calls provides a detailed behavioral trace of the application's logic, offering a higher resolution view than permissions alone [16]. Early machine learning models heavily relied on these sequences, often converting them into word vectors before feeding them into classifiers like Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs) [19]. Studies have shown that API calls and opcodes are among the most productive static features for malware detection [20]. Frameworks like IFDroid even built API call graphs for structural analysis, demonstrating the importance placed on this feature type [21]. As with permissions, API call features are also vulnerable to manipulation. The sheer volume of available APIs means that attackers can select a combination that appears benign while still

achieving their malicious goals [22]. Moreover, these features are susceptible to obfuscation techniques designed to alter the appearance of the code without changing its underlying function [23].

Intents and application components form another critical layer of feature representation. Intents are messaging objects that facilitate communication between different components of an Android application, such as Activities, Services, Broadcast Receivers, and Content Providers [6]. Intent filters specify the types of intents an activity or receiver can respond to, effectively defining how an application interacts with itself and other applications [13]. Analyzing these components and their relationships helps to understand the overall structure and interaction patterns of an app [6,10]. Some advanced detection methods integrate these features alongside permissions and API calls to create a more holistic view [24]. For example, a framework might flag an application if it declares a broadcast receiver that listens for boot-complete events but does not require internet access, a pattern inconsistent with many legitimate apps [25]. The combination of these three feature types—permissions, API calls, and intents—formed the bedrock of feature-centric malware analysis for years, powering numerous research projects and commercial tools [6,26].

Despite their widespread adoption, the reliance on hand-crafted features presents a fundamental weakness: brittleness. These features are symbolic representations that lack semantic understanding. Machine learning models trained on them learn statistical correlations between feature lists and malware labels, but they do not comprehend the contextual meaning or the overall behavioral intent of the application [27]. This limitation makes them highly susceptible to evasion through code obfuscation. Obfuscation is a deliberate act of making code difficult to understand, primarily to thwart reverse engineering and automated analysis [28]. Attackers employ several common techniques to defeat static analysis based on hand-crafted features. Control-flow flattening restructures the program's logic, scattering related code blocks throughout the application and making it difficult for analyzers to reconstruct a coherent flow [29]. While this complicates analysis, it does not inherently alter the sequence of API calls, though it can obscure the context in which they occur.

More direct attacks target the features themselves. String encryption and encoding involve encrypting or encoding strings within the application's code, including hardcoded API names, URLs, and sensitive values [30,31]. When an analyst or a simple parser encounters an encrypted string, it appears as gibberish, rendering the feature useless for analysis [31]. Similarly, API renaming replaces standard, descriptive API method names with meaningless, generic identifiers (e.g., 'a', 'b', 'method_1') [28]. This breaks the link between the decompiled code and the official Android documentation, preventing simple parsers from identifying potentially malicious functions. Another common technique is the insertion of junk code, non-functional code fragments added to increase the application's size and complexity, thereby confusing heuristic engines that rely on fixed patterns [23]. A large-scale study confirmed that these obfuscation techniques significantly impact the effectiveness of anti-malware products, often causing a sharp decline in detection rates [32,33]. The Drebin dataset, containing 5560 malware samples from 2014, serves as a historical benchmark but also highlights the age of this feature-based approach in the face of modern, sophisticated obfuscation [12,34]. Ultimately, the classical paradigm reached a point where its reliance on explicit, brittle features created a reactive security posture, constantly playing catch-up with evolving evasion tactics [1]. This led to the search for a more robust and adaptive solution, one capable of learning meaningful representations directly from the data itself.

2.2. Deep Learning and the Advent of Sequential Models

The limitations of classical static analysis, particularly its vulnerability to obfuscation and its reliance on manually crafted, brittle features, catalyzed a significant shift towards deep learning (DL) approaches [35,36]. The core premise of DL is its ability to automatically learn hierarchical representations of data from raw inputs, obviating the need for extensive feature engineering [37]. In the context of Android malware detection, this meant moving away from counting permissions and listing API calls towards training neural networks to discover complex patterns and latent structures directly within the application's code or its high-level representations [38]. Early DL models, such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), were among the first to be adapted for this task, marking a pivotal step in the evolution of malware detection methodologies [35,39].

Early deep learning approaches adapted CNNs to opcode/API sequences for local pattern capture [39]. However, their limited receptive fields and reliance on fixed-size inputs prevented effective modeling of temporal and cross-component behavioral dependencies [5,36]. To capture these longer-range patterns, researchers subsequently adopted recurrent architectures, particularly LSTMs and GRUs [35], which maintain an internal hidden state to preserve sequential context across API call streams [17]. However, standard RNNs, including LSTMs, are not without their own set of challenges. Their inherent sequential processing nature makes them difficult to parallelize, leading to slow training times compared to other architectures [5]. More critically, they suffer from the vanishing gradient problem, which hampers their ability to remember information over very long sequences, limiting their capacity to capture extremely long-range dependencies that might be present in large, complex applications [36]. This bottleneck in learning global context paved the way for the next major breakthrough in the field: the introduction of attention mechanisms embodied in the Transformer architecture.

The advent of Transformers revolutionized the field of Natural Language Processing (NLP) and quickly found its way into cybersecurity [40]. The key innovation of the Transformer is the self-attention mechanism, which allows the model to weigh the importance of different words (or tokens) in a sequence dynamically, regardless of their position [40]. Instead of processing data sequentially, Transformers process the entire input sequence in parallel, making them vastly more efficient to train [41]. When applied to malware detection, this meant that instead of being forced to process an API call sequence from start to finish, the model could simultaneously consider the relationship between every pair of API calls in the sequence. For instance, when analyzing a particular API call, the attention mechanism could determine that it is strongly correlated with a call made hundreds of lines earlier, giving that earlier call a high weight in its contextual representation [42]. This ability to capture global, long-range dependencies shattered the limitations of RNNs and provided a much richer, more nuanced representation of an application's behavior. The transition from RNNs to Transformers marked a profound change in feature representation, moving from a simple sequence of tokens to a set of dense, contextual embeddings where each token's representation is informed by the entire sequence [41]. This deeper level of semantic understanding laid the groundwork for the development of highly accurate models that could begin to reason about an application's intent rather than just matching patterns. The success of this paradigm was built on adapting pre-trained language models, originally developed for human languages, to the unique syntax and semantics of programming languages and bytecode, treating code as a distinct kind of text [43,44]. This set the stage for the next phase of evolution, where these powerful models would be further refined and specialized for the task of malware analysis.

2.3. Transformer-Based Architectures for Semantic Understanding

The adaptation of Transformer-based architectures from Natural Language Processing (NLP) to cybersecurity represents a significant leap forward in the semantic understanding of Android applications [40]. Models like BERT (Bidirectional Encoder Representations from Transformers) and its derivatives leverage the power of self-attention to generate deep, contextual representations of code, moving beyond the superficial pattern matching of earlier methods [44]. This shift allows the models to grasp the nuanced relationships between code elements, making them more resilient to certain forms of obfuscation that render simpler models ineffective [45]. The core idea is to treat various aspects of an APK—such as its manifest file, API call sequences, or even opcodes—as a form of structured text, which can then be processed by a pre-trained language model [46,47]. This approach fundamentally alters the feature representation pipeline, replacing hand-crafted features with learned, dense vector embeddings that encapsulate semantic meaning.

One of the pioneering applications in this area is MalBERT, which specifically fine-tunes the BERT architecture on Android manifest data [43,44]. The `AndroidManifest.xml` file is a rich source of information, detailing requested permissions, declared components (activities, services, etc.), and intent filters [10,48]. MalBERT treats this XML content as a continuous sequence of tokens and utilizes BERT's bidirectional encoder to understand the context of each element. For example, it can learn that the combination of requesting a high-risk permission like `REQUEST_INSTALL_PACKAGES` alongside a specific activity name is far more suspicious than seeing the permission in isolation. This contextual awareness, enabled by the self-attention mechanism, allows MalBERT to achieve high detection accuracy, with some implementations reporting results over 97% by capturing these subtle but critical relationships [44]. The model learns a vector embedding for each token in the manifest, where the embedding is not just a one-hot identifier but a dense vector influenced by all other tokens in the document. This moves the representation from a discrete, symbolic space to a continuous, semantic one, providing a much richer substrate for classification.

Beyond manifest analysis, other Transformer variants have been applied to different code representations. RoBERTa, an optimized version of BERT with more robust pre-training objectives, has been utilized to analyze opcode sequences [44]. Opcodes are the lowest-level representation of an application's logic, and treating them as a textual sequence allows the model to learn malicious patterns at a fundamental level [44]. Studies have shown that RoBERTa can outperform other models in identifying malicious patterns within these sequences, highlighting the effectiveness of robust pre-training strategies for security tasks [44]. Similarly, some research has explored encoding raw memory bytes as textual tokens, allowing a transformer to process the application's memory image directly and learn from byte-level patterns without relying on handcrafted features [41]. This demonstrates the versatility of the Transformer architecture in handling various levels of code representation. To enhance feature discriminability, some frameworks incorporate contrastive learning techniques, which encourage the model to pull embeddings of malicious samples closer together and push benign samples apart, further strengthening the quality of the learned representations [49]. These models collectively showcase the power of transformers to distill complex, multi-faceted information from an application into a compact, semantically rich vector that is far more informative than a simple list of features.

However, the computational cost and latency of full-sized Transformer models like BERT pose a challenge for real-time, large-scale deployment, such as scanning millions of applications in an app store [50]. This has spurred the development of distilled models, which aim to retain the performance of their larger “teacher” counterparts while being significantly smaller and faster. DistilBERT is a prominent example of this approach [44].

It uses a knowledge distillation technique to train a smaller model that mimics the output of the original BERT model. Research has shown that DistilBERT can achieve up to 97% of the performance of the teacher model while reducing the number of parameters and inference time by approximately 40–60% [44]. This makes it an attractive option for environments with limited computational resources or strict latency requirements, such as mobile endpoints or high-throughput scanners [51]. The viability of lightweight transformers like DistilBERT for real-time malware detection has been demonstrated, balancing accuracy, efficiency, and interpretability [51]. The findings suggest that for many practical scenarios, distilled models offer a compelling compromise, delivering near-state-of-the-art performance with substantially lower overhead. However, a debate persists regarding the trade-offs involved in distillation. While some argue that distillation inevitably leads to a loss of nuanced feature-detection capabilities required for highly sophisticated obfuscation schemes, others contend that with proper tokenization and fine-tuning, distilled models can perform exceptionally well, sometimes even outperforming larger models on specific tasks [44,51]. This ongoing discussion underscores the delicate balance between model complexity, performance, and efficiency that continues to shape the field. The integration of graph-based methods, which represent an APK as a graph of entities and their relationships, often complements these sequential models, providing a richer, multimodal view of the application that can further enhance detection resilience [21,52,53].

2.4. Generative LLM Frameworks for Reasoning and Interpretability

The latest frontier in static Android malware detection involves the application of Large Language Models (LLMs), such as those in the GPT and Llama families, which represent a paradigm shift from classification to generative reasoning [54,55]. Unlike previous models that output a binary label (malicious/benign), LLMs possess the capability to process prompts and generate human-readable text, opening the door to unprecedented levels of interpretability and explainability [56]. This generative capacity promises not only to match or exceed the accuracy of existing deep learning models but also to articulate the rationale behind their decisions, addressing the long-standing “black box” problem that plagues many machine learning-based security systems [56,57]. Instead of simply classifying an application, an LLM can be prompted to produce a natural language summary of its potential behavior, highlighting suspicious activities, connections between disparate code sections, and other indicators of malicious intent [46].

Several innovative frameworks have emerged to harness the power of LLMs for malware analysis. One such framework is LLM-MalDetect, which is explicitly designed to improve LLM-based APK analysis by modeling semantic dependencies within an application [46]. This framework leverages the LLM’s reasoning capabilities to generate a detailed semantic report of the APK, which can then be analyzed to make a detection decision [46]. By focusing on generating a reasoned explanation rather than just a label, LLM-MalDetect aims to provide users with greater confidence and transparency in the detection process [46]. Another approach, exemplified by the AppPoet framework, uses LLMs as a preprocessing step to convert traditional static features (like permissions and API calls) into natural-language descriptions before passing them to a classifier [58]. This enhances interpretability by making the input features understandable to human analysts, bridging the gap between the model’s internal logic and human comprehension. Recognizing the practical challenges of applying massive LLMs directly to large-scale datasets, context-driven frameworks like LAMD (LLM-based Android Malware Detector) have been proposed [59]. LAMD incorporates modules to isolate and extract key security-relevant context from an application, allowing the LLM to focus its computational power on the most salient parts of the code, thereby improving efficiency and effectiveness [59]. These

frameworks illustrate a growing trend towards integrating LLMs not necessarily as standalone classifiers, but as powerful analytical engines that augment the detection pipeline with deep semantic reasoning.

This move towards generative reasoning introduces a new and complex set of performance trade-offs, primarily centered around the tension between interpretability and reliability. On one hand, the ability of LLMs to generate explanations is a significant advancement, promising to demystify the detection process and enable more informed security responses. On the other hand, the reliability of these models in a high-stakes security context is a major concern. A critical challenge is the phenomenon of “hallucination,” where an LLM generates plausible-sounding but factually incorrect or nonsensical analysis [60]. In the context of malware detection, a hallucination could lead to a false positive (flagging a benign app) or, more dangerously, a false negative (overlooking a malicious one). Recent studies have highlighted that LLMs can indeed suffer from higher false positive rates compared to more specialized static analysis tools, and their outputs require careful validation [54,61]. The risk of generating harmful or misleading content is a significant safety consideration that must be addressed before these models can be trusted for fully automated decision-making [62].

Furthermore, the adoption of LLMs brings substantial computational and economic costs. The large parameter count of these models translates into high memory requirements and significant inference latency, raising serious questions about their feasibility for real-time scanning of vast app repositories [47,50]. While smaller, fine-tuned versions of LLMs are being explored, there remains a notable performance gap between proprietary, closed-source models (which often have good training data and optimization) and their open-source counterparts in security-related tasks [54]. This creates a barrier to entry for organizations and researchers who cannot afford access to expensive proprietary APIs. To address the challenges of evaluating these nascent technologies, the research community is developing specialized benchmarking frameworks. Cama (Code LLMs for Android Malware Analysis) and MalEval (Malware Evaluation) are two such initiatives designed to systematically assess the effectiveness, robustness, and reliability of Code LLMs across various tasks [61,63]. These frameworks aim to establish standardized evaluation methodologies, providing a common ground for comparing different models and ensuring reproducibility—a critical need given the recognized flaws in many existing software protection evaluations [64]. The current consensus suggests a hybrid approach, where LLMs are used for feature enrichment, threat intelligence generation, and providing explanations, rather than serving as the sole arbiters of trustworthiness. This mitigates the risks associated with hallucinations and unreliability while still leveraging the unique strengths of generative AI in understanding complex codebases.

2.5. Comparative Analysis of Performance Trade-Offs

The methodological evolution from classical static analysis to Transformer-based models and finally to generative LLMs reflects a continuous effort to enhance detection accuracy and resilience against increasingly sophisticated evasion techniques. However, each successive generation of technology introduces a distinct set of performance trade-offs, primarily concerning detection accuracy, computational efficiency, and robustness against obfuscation. A systematic comparison of these paradigms reveals a clear progression in capability but also highlights the inherent compromises between power, speed, and resource consumption. The following analysis synthesizes the available information to provide a comparative overview of these critical trade-offs, as summarized in Table 1.

Table 1. Comparison of malware detection paradigms in static analysis.

Paradigm	Key Technologies/Models	Primary Feature Representation	Strengths	Weaknesses
Classical Static Analysis	Drebin, SVM, Random Forests	Hand-crafted: Permissions, API Calls, Intents [7,8], Network Addresses [10]	High computational efficiency, scalable, interpretable feature set (to humans) [3,4]	Vulnerable to obfuscation (e.g., string encryption, API renaming) [28,31], poor generalization to zero-day threats [32]
Deep Learning	CNNs, RNNs (LSTM)	Sequences of API calls, opcodes, or visual representations of bytecode [17,39]	Automatic feature extraction, captures local (CNN) and sequential (RNN) patterns better than classical methods [37]	Struggles with long-range dependencies (RNNs), slow training (RNNs), and can be brittle to structural changes in code [36]
TransformerBased	MalBERT, RoBERTa, DistilBERT [44]	Dense, contextual embeddings from API call/opcode sequences or manifest text [41]	Captures global, long-range dependencies via self-attention, improved resilience to some obfuscations, high accuracy [44,65]	Full models are computationally expensive; distilled versions may sacrifice nuance [51]
Generative LLMs	LLM-MalDetect, LAMD, AppPoet [46,58,59]	Natural language summaries of application behavior generated by an LLM [46]	Unmatched interpretability/explainability, strong reasoning capabilities, high potential accuracy [46,56]	Very high computational cost and latency, risk of hallucination (false positives/negatives), reliability concerns [50,60]

In terms of detection accuracy, the trajectory is clear. Classical methods like Drebin, while efficient, exhibit declining performance as obfuscation becomes prevalent [32]. Deep learning models like CNNs and RNNs offered improvements by learning more complex patterns from raw data, but their accuracy was often limited by their architectural constraints [35,39]. Transformer-based models represent a significant leap in accuracy. MalBERT, for instance, achieves over 97% accuracy on some benchmarks by leveraging contextual embeddings from manifest data [44]. The choice of dataset also plays a crucial role in reported accuracy figures. Evaluations on classic datasets like Drebin show varying results depending on the model, while newer datasets like CICAndMal2017 may yield different outcomes, highlighting the need for standardized benchmarks [66,67]. Generative LLMs are positioned to deliver the highest accuracy, with frameworks like LLM-MalDetect claiming performance comparable to deep learning models, but with the added benefit of reasoning [46]. However, this potential is tempered by the aforementioned risk of hallucination, which can artificially inflate or deflate perceived accuracy depending on how errors are measured [61].

Computational efficiency and inference latency stand as the most significant counterweights to the increasing accuracy of these models. Classical methods are the undisputed champions of efficiency, capable of scanning thousands of applications per minute on modest hardware [3]. Deep learning models, especially RNNs with their sequential processing, are considerably slower to train and can have high latency during inference [5]. Transformers offer better parallelization, but their large parameter sizes result in high memory usage and computational demands [50]. This is where distilled models like DistilBERT become strategically important. By reducing model size and inference time by up to 60% while retaining most of the performance, they offer a viable path to deploying powerful semantic models in resource-constrained environments [44,51]. Generative LLMs, however, operate on a completely different scale of resource consumption. The largest models require powerful GPU clusters and have latencies that can range from seconds to minutes per application, making them impractical for large-scale, real-time scanning [47,50]. Their use is currently more suited to in-depth forensic analysis of suspicious applications flagged by faster, upstream scanners.

The most critical trade-off in the context of modern malware is resilience to obfuscation. Classical methods are notoriously fragile; a single string encryption or API rename can break the feature extraction pipeline and evade detection [28,31]. Deep learning models showed moderate improvement, but their effectiveness was still tied to the integrity of the input sequence. Transformer-based models demonstrate a notable increase in resilience. Because they learn contextual embeddings, they are less dependent on the specific names of APIs or opcodes. If an API is renamed, the model may still recognize it based on its surrounding context, preserving the semantic signal [41]. Similarly, the attention mechanism can help down-weight irrelevant or noisy code segments, such as those introduced by junk code insertion [41]. However, their resilience to more complex structural obfuscations like control-flow flattening is less certain, as this technique scrambles the very sequences the model relies on [29]. Generative LLMs are theoretically the most resilient, as their strength lies in high-level reasoning about an application's purpose, not just the surface-level features. An LLM might analyze a series of obfuscated API calls and, based on the overall pattern and logical flow, deduce the application's true intent, bypassing the need to understand each individual obfuscated component [56]. Nevertheless, the arms race continues, with new adversarial attack frameworks like LAMLAD now being developed to exploit the generative and reasoning capabilities of LLMs themselves, indicating that no defense is ever truly permanent [60].

3. Materials and Methods

3.1. Dataset Selection and Programmatic Sampling Protocol

The foundational step in constructing any robust machine learning-based malware detection system is the acquisition of a high-quality, well-labeled dataset. This section details the scientific methodology employed for data acquisition, focusing on the selection of AndroZoo as the primary repository, programmatic retrieval of application packages (APKs) via its API, and the rigorous process of establishing ground-truth labels. AndroZoo was selected as the data source due to its established reputation as a comprehensive, publicly accessible, and continuously updated collection of Android applications. Its value for rigorous scientific inquiry stems from three key attributes: (i) Scale: housing millions of distinct APKs, it provides a diverse sample pool that mitigates overfitting to dataset-specific idiosyncrasies; (ii) Metadata Enrichment: each entry is augmented with VirusTotal (VT) scan reports, permission lists, and static analysis features; (iii) Reproducibility: leveraging a public, well-documented resource with stable identifiers (SHA-256 hashes) enables independent validation and direct replication of our experimental corpus.

To ensure full transparency and reproducibility, we constructed the final dataset of 12,000 APKs (6000 benign, 6000 malicious) using a deterministic, script-driven pipeline applied to the official AndroZoo metadata archive (latest.csv.gz). Based on AndroZoo's standard schema, column 8 corresponds to the VirusTotal detection count. Malicious samples were selected using the criterion $vt_detection \geq 25$, ensuring high-confidence malicious labeling while excluding bloated or repackaged outliers. Benign samples required exactly $vt_detection == 0$ and, further verified against official distribution channels (Google Play, F-Droid). Samples with 1–24 VT detections were explicitly excluded to prevent ambiguous ground-truth labeling that could degrade classifier calibration. SHA-256 hash-level deduplication was applied prior to random sampling to guarantee that no identical binaries appeared multiple times across the corpus. From each filtered subset, exactly 6000 unique hashes were drawn using cryptographic shuffling (shuf-n 6000), yielding a balanced corpus. All filtering, exclusion, and sampling steps were executed via version-controlled shell scripts, ensuring that any researcher with access to the same AndroZoo snapshot can reproduce the exact dataset composition.

The temporal distribution of the sampled applications reflects both the historical evolution of the Android ecosystem and the application of our filtering criteria. Benign applications span from 2011 to 2025, with pronounced concentrations in 2013–2014 ($n = 3300$), corresponding to the period of rapid Android market expansion and Google Play Store growth. A secondary peak occurs in 2016 ($n = 1003$), followed by a notable decline in subsequent years, likely attributable to stricter Google Play review policies and enhanced automated screening mechanisms introduced post-2017. In contrast, malicious samples exhibit a distinct temporal pattern, with minimal presence before 2015 and a dramatic surge between 2016 and 2020 ($n = 4837$, representing 80.6% of all malware samples). This concentration aligns with documented periods of intensified Android malware campaigns, particularly the rise in banking trojans, SMS fraud, and ransomware families during this era. The sharp decline in malware samples post-2020 ($n = 389$ for 2021–2024) is primarily attributable to our stringent $VT \geq 25$ consensus threshold, as recently uploaded samples have not yet accumulated sufficient AV scan history to meet this criterion. Additionally, the 1 MB size constraint disproportionately filters out modern applications, which tend to be larger due to increased library dependencies and resource bundling. This temporal distribution inherently supports evaluation across different Android API levels and security paradigms, while the stratified 5-fold cross-validation protocol ensures that each fold preserves this temporal heterogeneity, mitigating time-based sampling bias.

The “ground truth” used in this study is derived from a consensus among multiple AV vendors aggregated via VirusTotal, rather than a single source. This multi-engine approach reduces the risk of label noise inherent to any individual AV engine’s detection logic. For benign samples, the combination of zero VT detections and verification against trusted repositories provides strong evidence of non-malicious intent.

3.2. Dataset Construction

The core of the static analysis phase involves extracting a quantitative set of features from each APK that can serve as input for a machine learning classifier. The methodology employs AndroPyTool, a specialized framework designed for the automated extraction of a wide array of static features from Android applications. The scientific rationale for selecting AndroPyTool lies in its ability to systematically generate a numerical, tabular representation of an app’s characteristics, which is highly suitable for training traditional machine learning models. It acts as a powerful feature engineering tool, abstracting away the complexities of low-level code inspection and providing a structured output, often in JSON, that can be readily consumed by classifiers. The literature extensively documents its use in creating large-scale datasets and frameworks for malware detection, underscoring its utility and reliability in the research community.

By using AndroPyTool, the methodology ensures a consistent and comprehensive approach to feature extraction across all 12,000 samples in the dataset. Based on its documented functionality and usage in related research, AndroPyTool extracts several key categories of static features, as detailed in Table 2, that are indicative of an application’s behavior and potential for malicious activity. These features provide a statistical overview of the app’s components and interactions, forming a foundational layer for malware classification.

Table 2. Feature Categories in Static Android Malware Analysis: Technical Descriptions and Security Implications.

Feature Category	Description	Scientific Rationale
Permissions	Declared Android system access rights (e.g., INTERNET, READ_CONTACTS) listed in AndroidManifest.xml.	Permission declarations define the app's authorized attack surface and are critical for static risk scoring; absence may indicate benign minimalism or deliberate evasion to bypass permission-based malware detectors [68]. Discrepancies between declared and used permissions are a known indicator of malicious behavior, as verified through inter-component data flow analysis [69].
Opcodes	Dalvik bytecode instructions (e.g., return-void, invoke-direct) extracted from .dex files.	Opcode n-grams and distributions serve as robust features for ML-based malware classification, as malicious code often exhibits distinct instruction patterns due to obfuscation, packing, or payload injection [70]. Sparse opcode profiles may signal stub classes or dynamically loaded code, which can be detected through hybrid opcode feature vector analysis [71].
API Calls	Static detection of invoked Android framework or third-party methods (e.g., getDeviceId(), executeHttpRequest()).	API call graphs enable behavior-based static analysis; calls to sensitive APIs (e.g., telephony, SMS, cryptography) are strong predictors of malicious intent and form the basis for rule-based and ML detection systems [72]. Absence may indicate runtime reflection or native code execution, which are common evasion techniques requiring hybrid static-dynamic analysis to detect [73].
Strings	Literal text values embedded in the binary (e.g., URLs, keys, commands) extracted via static disassembly.	Hardcoded strings provide high-value indicators for signature matching, C2 domain detection, and credential leakage analysis; their absence often implies string encryption or dynamic generation, a common evasion technique in advanced malware [74]. Deobfuscation tools demonstrate that encrypted strings can be recovered to expose malicious indicators [75].
App Components	Core Android building blocks (Activities, Services, Receivers) declared in the manifest governing UI, background tasks, and event handling.	Component declaration patterns inform execution context modeling; minimal or empty component sets suggest non-standalone artifacts (e.g., plugins, libraries) or stripped droppers, while unexpected component exposure can enable inter-app attacks [76]. Android 12+ security policies now require explicit android:exported declarations to mitigate component-hijacking risks [77].
Intents	Inter-component communication messages defining actions, data URIs, categories, and explicit/implicit targets.	Intent analysis maps data flow and component coupling; malformed or overly permissive intent filters are associated with component hijacking, privilege escalation, and unauthorized data exfiltration vulnerabilities [78]. Intent parameter injection and redirection attacks remain prevalent in Google Play applications, emphasizing the need for intent validation in static analysis pipelines.
API Packages	Aggregated Java/Kotlin package namespaces of invoked framework or third-party libraries (e.g., android.telephony, java.net, javax.crypto) extracted from .dex bytecode references.	Package-level aggregation provides a higher-order semantic view of an app's functional scope, reducing feature dimensionality while preserving behavioral intent. Malware frequently exhibits concentrated usage of specific package families that diverge from benign baselines. Package distributions are resilient to method-level obfuscation and enable efficient feature engineering for ML classifiers, as they capture domain-specific behavior without relying on fragile individual API signatures [79]. Empty or anomalous package profiles may indicate dynamically loaded code, reflection-based evasion, or stripped payloads.

To construct a binary feature matrix compatible with standard machine learning algorithms, begin by initializing an empty global feature set F and iterating over all 12,000 applications. For each APK, extract the complete set of observed features across all categories (permissions, opcodes, API calls, strings, components, intents, and API packages) using static analysis frameworks such as AndroPyTool or Androguard. Incrementally union these observations into F such that $F = \bigcup_{i=1}^{12,000} F_i$, where F_i denotes the features present in application a_i . Once F is finalized and indexed (e.g., $F = \{f_1, f_2, \dots, f_n\}$), initialize a binary matrix $x \in \{0, 1\}^{12,000 \times n}$. For each application a_i , populate its corresponding row x_i by setting $x_{ij} = 1$ if feature f_j was observed during extraction, and $x_{ij} = 0$ otherwise, irrespective of its corpus-wide frequency. This strict presence-only encoding yields a fixed-dimensional, tabular representation that directly satisfies the input specifications of standard supervised learning algorithms (e.g., logistic regression, support vector machines, random forests, and gradient-boosted trees), which require numerically homogeneous, vectorized features without domain-specific structural preprocessing. Although this approach increases dimensionality and sparsity, storing x in a compressed sparse row (CSR) format maintains computational tractability. Finally, append a ground-truth malware label $y_i \in \{0, 1\}$ for each sample and export the dataset in a reproducible format (e.g., CSV or HDF5), accompanied by a feature dictionary mapping each column index j to its semantic descriptor. This methodology ensures an algorithm-agnostic, minimally processed representation that integrates seamlessly into conventional ML pipelines, while deferring dimensionality reduction or feature weighting to model-intrinsic regularization or post hoc selection techniques.

To ensure strict adherence to the input constraints of transformer-based architectures—specifically the maximum sequence length limitation (e.g., 512 tokens for BERT-base)—a data-driven feature selection methodology was employed to construct the per-application input sequences. Following the extraction of static indicators across seven orthogonal categories (Permissions, API Calls, API Packages, Intents, Opcodes, Strings, and App Components) via AndroPyTool, a corpus-wide univariate and multivariate feature ranking procedure was executed to quantify the discriminative salience of each feature with respect to the malware classification objective. Specifically, established feature selection algorithms—including Information Gain (IG), chi-square (χ^2) statistical testing, Mutual Information (MI), and L1-regularized logistic regression—were applied to the binary presence matrix to compute relevance scores for all observed features. The top- k features, where k is calibrated to accommodate tokenization overhead (e.g., special tokens [CLS], [SEP], and subword expansion under WordPiece encoding), and were retained for each application, thereby ensuring that the resulting token sequence remains within the transformer’s contextual window while maximizing informational density.

Each selected feature was subsequently serialized according to the typed schema `<CATEGORY> <semantic_identifier>` (e.g., `PERMISSION android.permission.SEND_SMS`, `API_CALL android.telephony.SmsManager.sendMessage`), preserving semantic interpretability and enabling direct tokenization via pre-trained subword vocabularies. This algorithmic selection strategy offers several methodological advantages over heuristic or priority-based truncation: (i) it grounds feature retention in empirical statistical evidence rather than domain-assumed risk priors, thereby mitigating confirmation bias; (ii) it adapts dynamically to the discriminative structure of the specific dataset, enhancing generalizability across diverse malware families; and (iii) it maintains reproducibility, as the selection pipeline and ranking criteria are explicitly documented and parameterized. The resulting corpus constitutes a structurally homogeneous yet semantically rich dataset, wherein each instance is represented as a token sequence optimized for attention-based modeling while retaining backward compatibility with conventional tabular classifiers through deterministic parsing of the typed feature identifiers. This approach thus reconciles

the representational demands of deep sequence models with the rigor, transparency, and comparability standards expected in empirical software security research.

4. Experimental Evaluations

Experiments were conducted on two distinct hardware configurations to ensure comprehensive evaluation across different computing environments. The first setup utilized an Intel Core i5 processor with 40 GB RAM and an NVIDIA GeForce RTX 3050 Ti GPU, running Windows 11. The second setup employed an Intel Xeon processor paired with an NVIDIA H100 GPU, operating on a Linux (Debian) environment.

In both configurations, the Python v.3.12 programming language was used with the core libraries, ensuring consistency in the analytical workflow. The AndroZoo dataset was used for experimentation across both platforms.

The following sections demonstrate the detailed breakdown of experimental evaluation across five classifiers. Each of the individual classifier sections describes the settings used and outcomes of the evaluations, with platform-specific considerations noted where applicable. Later in the section, we discuss the comparative results and performance observations across the two hardware environments.

4.1. Evaluation Metrics

To rigorously assess the predictive performance of the proposed model, we employed a comprehensive suite of classification metrics: Accuracy, Precision, Recall, F1-Score, and False Positive Rate (FPR). Let TP , TN , FP , and FN denote the number of true positives, true negatives, false positives, and false negatives, respectively, as derived from the model's confusion matrix.

Accuracy quantifies the overall proportion of correctly classified instances across both classes:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Precision measures the reliability of positive predictions by calculating the proportion of predicted positives that are truly positive:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

Recall (also termed Sensitivity or True Positive Rate) assesses the model's capacity to correctly identify all actual positive instances:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

The F1-Score provides the harmonic mean of Precision and Recall, offering a balanced evaluation that is particularly robust in the presence of class imbalance:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

To ensure reliable and reproducible estimation of model generalization, we employ stratified K-fold cross-validation throughout the experimental pipeline. This approach guarantees that each fold preserves the original 1:1 benign-to-malware class ratio, thereby mitigating sampling bias and ensuring statistically consistent validation splits. The evaluation is configured with $K = 5$ folds and a fixed random seed (`random_state = 42`) to guarantee deterministic partitioning and full reproducibility. In each iteration, the model is trained on four folds and evaluated on the held-out fold, yielding out-of-sample predic-

tions and calibrated probabilities for leakage-free performance estimation. Classification metrics—Accuracy, Precision, Recall, F1-Score—are computed per fold, and final results are reported as macro-averaged means $\pm$ standard deviations across five folds. This protocol is consistently applied to classical ML baselines, Transformer fine-tuning, and LLM-based adaptation stages, effectively preventing data leakage while minimizing overfitting risks in high-dimensional feature spaces.

To further ensure methodological rigor and prevent optimistic bias in performance estimation, several safeguards against data leakage and dataset contamination were enforced throughout the experimental pipeline. First, duplicate APK samples were eliminated through SHA-256 hash verification to ensure that no identical applications appeared multiple times across the corpus. Second, train-validation separation was maintained at the APK level under a stratified 5-fold cross-validation protocol, guaranteeing that samples used for evaluation remained entirely unseen during training. To mitigate family-level leakage, malware family distributions were carefully monitored across folds to reduce the risk of closely related variants appearing simultaneously in both training and validation partitions. In addition, potential version-level contamination was minimized by filtering redundant or near-identical application versions whenever applicable. Finally, all pre-processing operations—including feature selection, ranking, normalization, and sequence construction—were performed independently within each training fold and subsequently applied to the corresponding validation fold, thereby preserving full train/test independence and preventing indirect information transfer between partitions.

Statistical Significance Testing

To assess whether observed performance differences were statistically meaningful, all metrics were reported as macro-averaged mean $\pm$ standard deviation across stratified 5-fold cross-validation (`random_state = 42`). Pairwise comparisons were performed between the Random Forest baseline and each advanced detection paradigm. Depending on the distribution of fold-wise performance differences, either paired two-tailed *t*-tests or Wilcoxon signed-rank tests were employed. To account for multiple comparisons, the Bonferroni correction was applied across the four primary comparisons, resulting in an adjusted significance threshold of $\alpha_{\text{adj}} = 0.0125$. The results of the statistical comparisons are reported in Table A1, including mean F1-scores, performance differences relative to the Random Forest baseline, selected statistical tests, and corresponding *p*-values. These analyses complement the reported performance metrics by evaluating whether observed numerical differences reflect statistically reliable improvements.

4.2. Standard ML Algorithms

4.2.1. Feature Selection

To address the high-dimensional and heterogeneous nature of static APK feature spaces, this study adopts a hybrid ensemble feature selection strategy that systematically integrates filter, wrapper, and embedded methods. Let $\mathcal{X} \in \mathbb{R}^{n \times d}$ denote the feature matrix with *n* APK samples and *d* initial features, and $\mathbf{y} \in \{0, 1\}^n$ the corresponding binary labels (benign/malware). The selection pipeline proceeds as follows: First, correlation-based redundancy reduction eliminates one feature from each pair (f_i, f_j) satisfying $|\rho_{ij}| > \tau$, where Pearson's correlation coefficient is computed as

$$\rho_{ij} = \frac{\sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)}{\sqrt{\sum_{k=1}^n (x_{ki} - \bar{x}_i)^2} \sqrt{\sum_{k=1}^n (x_{kj} - \bar{x}_j)^2}}, \tau = 0.95$$

Second, three complementary selection mechanisms are applied in parallel: (i) Univariate statistical screening ranks features by the ANOVA F-statistic $F_i = \frac{\text{Var}_{\text{between}}(f_i)}{\text{Var}_{\text{within}}(f_i)}$ and mutual information $I(f_i; \mathbf{y}) = \sum_{x \in f_i} \sum_{y \in \mathbf{y}} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$, capturing both linear separability and non-linear dependencies; (ii) Tree-based importance estimation employs a Random Forest ensemble $\mathcal{T} = \{T_1, \dots, T_M\}$ to compute Gini importance for feature f_i :

$$\text{Imp}(f_i) = \frac{1}{M} \sum_{m=1}^M \sum_{t \in T_m} \mathbb{I}(t \text{ splits on } f_i) \cdot \frac{N_t}{N} \cdot \Delta \text{Gini}(t)$$

where N_t denotes samples at node t , N the total training samples, and $\Delta \text{Gini}(t)$ the impurity reduction achieved by the split; (iii) L1-regularized selection solves the convex optimization problem:

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^d}{\text{argmin}} \left\{ \frac{1}{2n} \|\mathbf{y} - \mathcal{X}\beta\|_2^2 + \alpha \|\beta\|_1 \right\}$$

retaining features with $\hat{\beta}_i \neq 0$, where α is selected via 5-fold cross-validation. Third, to synthesize method-specific rankings into a robust consensus, scores are normalized to $[0, 1]$ via min-max scaling $\tilde{s}_m(f_i) = \frac{s_m(f_i) - \min(s_m)}{\max(s_m) - \min(s_m)}$ (with inversion for ranking-based methods) and aggregated through a weighted ensemble:

$$S_{\text{ens}}(f_i) = \sum_{m \in \mathcal{M}} w_m \cdot \tilde{s}_m(f_i), \mathcal{M} = \{\text{RF}, \text{uni}, \text{Lasso}\}, \mathbf{w} = [0.4, 0.3, 0.3]^\top$$

The final feature subset $\mathcal{F}^* = \{f_i \mid S_{\text{ens}}(f_i) \geq \theta_k\}$ comprises the top- k features by ensemble score. This methodological path was deliberately chosen for three principal reasons. First, Android malware datasets often contain thousands of correlated permissions, API calls, and structural metrics; relying on a single selection criterion risks overlooking context-dependent or interaction-driven signals critical for distinguishing obfuscated malware variants. Second, ensemble aggregation reduces the variance and method-specific bias inherent in individual selectors—e.g., filter methods ignore feature dependencies ($\mathbb{E}[\text{Var}(\hat{F}_{\text{ens}})] \leq \min_m \mathbb{E}[\text{Var}(\hat{F}_m)]$), while wrapper methods may overfit to the training distribution—thereby enhancing generalizability across diverse malware families. Third, by preserving a transparent, multi-criteria ranking process, the framework supports forensic interpretability: security analysts can trace selected features back to their methodological origins (e.g., a permission flagged by both F -test and Random Forest), facilitating hypothesis generation about malicious behavioral patterns. Therefore, this approach not only optimizes downstream classification performance but also aligns with the operational requirements of explainable, reproducible, and adaptive Android malware analysis.

4.2.2. Standard ML Algorithms and Results

To establish rigorous empirical baselines and systematically evaluate the discriminative capacity of the selected APK features, we implement a comprehensive classification pipeline comprising six canonical supervised learning algorithms: Random Forest (RF), Gradient Boosting (GB), Logistic Regression (LR), Support Vector Machine (SVM), Decision Tree (DT), and k -Nearest Neighbors (KNN). These models were deliberately chosen to span a wide spectrum of inductive biases—ranging from linear decision boundaries and probabilistic generative assumptions to non-parametric distance metrics and sequential ensemble aggregation—thereby mitigating model-specific failure modes and ensuring a thorough, unbiased assessment of feature utility across diverse learning paradigms. Prior to training, the feature matrix $X \in \mathbb{R}^{n \times d}$ undergoes zero-imputation for missing

entries and standardization via $z_i = (x_i - \mu)/\sigma$, which is essential for scale-sensitive learners (SVM, KNN, LR) to converge reliably and avoid dominance by high-variance features. Model evaluation employs stratified K -fold cross-validation ($K = 5$) with fixed randomization, partitioning the dataset into disjoint folds while preserving the original malware-to-benign class ratio. For each fold, the hypothesis is trained on the training partition and yields out-of-sample predictions and calibrated class probabilities, enabling leakage-free performance estimation.

Classification efficacy is quantified using macro-averaged accuracy, and per-class precision, recall, and F1-scores. These metrics collectively capture both overall discriminative power and class-stratified performance, which is critical in security contexts where the cost of false negatives (undetected malware) and false positives (benign apps flagged as malicious) is asymmetric and operationally significant. Accuracy provides a global measure of correct classifications, while AUC summarizes the model's ability to rank malicious samples above benign ones across all decision thresholds. Precision, recall, and F1-score further decompose performance at the class level, enabling targeted analysis of detection sensitivity versus alarm reliability.

This methodological trajectory is explicitly chosen for three interrelated reasons. First, Android malware detection operates in a high-dimensional, conceptually noisy, and often imbalanced feature space; deploying a heterogeneous suite of well-characterized classical algorithms establishes a statistically sound baseline that isolates feature quality from architectural overfitting or hyperparameter exploitation. Second, stratified cross-validation coupled with standardized preprocessing guarantees that performance estimates reflect true generalization capability rather than data leakage or scale-induced bias, which is particularly critical for margin-based (SVM) and distance-based (KNN) learners. Third, the explicit reporting of both aggregate and class-stratified metrics aligns with operational cybersecurity requirements: high recall minimizes undetected malware (reducing false negatives), while controlled precision limits false alarms that would overwhelm security analysts or trigger alert fatigue. By systematically benchmarking these computationally efficient, interpretable, and theoretically grounded models prior to deploying deep or black-box architectures, the pipeline adheres to the principle of parsimony, ensuring that any subsequent performance gains are attributable to genuine representational advances rather than optimization artifacts. This classification framework not only quantifies the discriminative power of the engineered APK features but also provides a transparent, reproducible, and statistically rigorous foundation for comparative algorithmic research in mobile security analytics.

The performance comparison, as detailed in Table 3, demonstrates that all six evaluated classifiers achieve highly robust and balanced results, with accuracy, precision, recall, and F1 scores consistently above 0.95. Random Forest yields the best performance (0.975 ± 0.003 F1-Score), closely followed by SVM (0.968 ± 0.003 F1-Score). The low standard deviations ($\sigma < 0.006$) across all models indicate stable performance under stratified 5-fold cross-validation, attributable to the balanced dataset (1:1 benign-to-malicious ratio) and macro-averaging protocol. Under these conditions, precision and recall naturally converge when false positive and false negative rates are symmetric, explaining the near-identical metric values observed across models.

The high performance of tree-based methods (Random Forest, Gradient Boosting, Decision Tree) along with linear (Logistic Regression) and instance-based (k-NN) approaches suggests that the underlying feature space possesses strong discriminative power and low noise levels. Moreover, consistent results across different algorithmic families highlight the inherent separability of the classes and the quality of the preprocessing steps (e.g., scaling, handling of missing values, hyperparameter tuning). This level of agreement

between metrics further confirms that all models generalize reliably, with no single model suffering from overfitting or underfitting. Any of these classifiers could potentially support real-world pipelines, with Random Forest offering a marginal yet notable advantage for this specific dataset.

Table 3. Performance comparison of standard ML algorithms.

Algorithms	Accuracy	Precision	Recall	F1-Score
Random Forest	0.975 ± 0.003	0.976 ± 0.003	0.974 ± 0.003	0.975 ± 0.003
SVM	0.968 ± 0.003	0.969 ± 0.003	0.967 ± 0.004	0.968 ± 0.003
Gradient Boosting	0.958 ± 0.004	0.958 ± 0.004	0.957 ± 0.004	0.958 ± 0.004
Decision Tree	0.956 ± 0.005	0.957 ± 0.005	0.955 ± 0.005	0.956 ± 0.005
Logistic Regression	0.955 ± 0.004	0.956 ± 0.004	0.954 ± 0.004	0.955 ± 0.004
K-NN	0.955 ± 0.005	0.956 ± 0.005	0.954 ± 0.006	0.955 ± 0.005

Note: Values are reported as mean ± standard deviation across stratified 5-fold cross-validation ($K = 5$, random_state = 42). All metrics are macro-averaged to ensure equal weighting of benign and malicious classes. Statistical significance testing protocol detailed in the Statistical Significance Testing Section. The near-identical precision/recall/F1 values arise from the balanced dataset (1:1 ratio) and symmetric error distributions under macro-averaging.

To contextualize the performance of the proposed Android malware detection approach, we conducted a comparative analysis against some studies in the literature. Table 4 summarizes the key characteristics and evaluation metrics of five representative works, alongside the results obtained in this study. The selected studies were chosen based on their methodological relevance, use of publicly available datasets (e.g., CICMalDroid 2020, AndroZoo, Drebin), and reporting of standard classification metrics (Accuracy, Precision, Recall, F1-Score). It should be noted that direct numerical comparison across studies must be interpreted with caution, as differences in dataset composition, feature extraction pipelines, preprocessing strategies, and validation protocols may influence reported performance. Nevertheless, this synthesis provides a meaningful benchmark for assessing the relative effectiveness of the proposed Random Forest-based framework under a hybrid feature representation scheme.

Table 4. Benchmarking of Android malware detection studies with standard ML algorithms.

Study	Dataset	Model	Feature Type	Accuracy	Precision	Recall	F1-Score
[80]	CICMalDroid 2020	XGBoost	Hybrid(dynamic + static)	0.9747	0.9703	0.9731	0.9716
[81]	AndroZoo	Random Forest	API Calls + Permissions	0.9651	-	-	-
[82]	Drebin + AndroZoo	Ensemble(RF, SVM, k-NN)	Permissions + API	0.9510	-	-	-
[83]	Drebin + AndroZoo	Random Forest	Permissions only	0.9159	0.9160	0.9160	0.9160
[84]	AndroZoo	Random Forest	Hybrid(API + Permission + System Call)	0.96	0.9300	0.9000	0.9100
This study	AndroZoo	Random Forest	All Static Features	0.975 ± 0.003	0.976 ± 0.003	0.974 ± 0.003	0.975 ± 0.003

Note: Results for “This study” are reported as mean ± standard deviation across 5-fold cross-validation. Due to heterogeneous datasets, temporal ranges, feature engineering pipelines, and validation protocols across cited works, direct numerical comparison or state-of-the-art claims are not statistically valid. This table serves as a qualitative context for methodological positioning.

As illustrated in Table 4, the proposed approach achieves competitive performance relative to existing methods, attaining an F1-Score of 0.9752 on the AndroZoo dataset using a Random Forest classifier with a comprehensive hybrid feature set. Notably, the consistency across all evaluation metrics (Accuracy = Precision = Recall = F1) suggests bal-

anced discriminative capability for both benign and malicious samples, mitigating potential biases toward the majority class. While Study [80] reports a marginally lower F1-Score (0.9716) using XGBoost on the CICMalDroid 2020 dataset, direct comparison is constrained by differences in dataset scale, malware family distribution, and feature dimensionality. Studies relying solely on permission-based features [83] exhibit comparatively lower performance ($F1 = 0.9160$), underscoring the added value of integrating multi-source static and dynamic indicators. Furthermore, the ensemble method in [82], despite leveraging dimensionality reduction, does not surpass the proposed single-model framework, highlighting the efficacy of thoughtful feature engineering over algorithmic complexity alone. These observations collectively support the validity of our hybrid feature extraction strategy and reinforce the suitability of Random Forest for high-dimensional, heterogeneous Android malware detection tasks. Future work will focus on cross-dataset generalization testing and adversarial robustness evaluation to further strengthen practical deployability.

4.3. Transformers Models

This section focuses on our novel input representation and sequence-handling strategies tailored to Android static analysis. To address the strict token-length constraints of encoder architectures (typically 512 tokens), we introduce a constraint-aware multi-level filtering pipeline that dynamically selects the most information-dense feature subset per APK while preserving semantic interpretability. When sequences exceed the token limit, we employ chunking with mean/max pooling to capture distributed malicious patterns without arbitrary truncation. The following subsections detail the implementation and empirical performance of BERT, RoBERTa, and DistilBERT under this unified framework.

4.3.1. Multi-Level Feature Filtering and Constraint-Aware Sequence Selection

As stated before, each APK-level JSON feature file was transformed into a textual document suitable for BERT-based classification. Static-analysis attributes, including permissions, API calls, API packages, intents, activities, services, receivers, and system commands, were extracted. Each feature was serialized as a line of text using a predefined semantic prefix, such as PERMISSION, API_CALL, or INTENT, followed by the corresponding feature value. This representation preserves the categorical identity of each feature while enabling the model to process static-analysis data as natural-language-like sequences. The resulting text files constitute BERT-ready documents, where each Android application is represented as a sequence of static behavioral tokens for downstream malware classification.

Transformer-based language models such as BERT impose a strict upper bound on input sequence length (typically 512 tokens). This constraint poses a significant challenge when representing Android applications as textual sequences derived from static analysis features, since the number of extracted features can vary substantially across applications. To address this limitation, a multi-level feature filtering and sequence selection strategy was employed. Specifically, for each Android application, multiple textual representations were generated by applying different feature filtering thresholds (e.g., top-N features or frequency-based filtering). These representations correspond to varying levels of feature granularity, ranging from highly detailed (long sequences) to aggressively filtered (short sequences). Formally, let $D_i = \{T_{i1}, T_{i2}, \dots, T_{ik}\}$ denote the set of candidate textual representations for application i , where each T_{ij} is associated with a token length L_{ij} computed using the tokenizer of the pre-trained BERT model (specifically, bert-base-uncased). The objective is to select an optimal representation T_i^* such that it maximizes the amount of retained information while satisfying the model's input constraint. The selection strategy is defined as follows:

Primary criterion (no truncation):

Among all candidate representations where $L_{ij} \leq 512$, the representation with the maximum token length is selected:

$$T_i^* = \arg \max_j \{L_{ij} \mid L_{ij} \leq 512\}$$

Fallback criterion (minimal overflow):

If no candidate representation satisfies the length constraint, the representation with the minimum token length exceeding 512 is selected:

$$T_i^* = \arg \min_j \{L_{ij} \mid L_{ij} > 512\}$$

This strategy ensures that, whenever possible, the selected sequence fully fits within the BERT input limit without requiring truncation, thereby preserving the maximum amount of semantic information. In cases where all candidate sequences exceed the limit, the shortest sequence is chosen to minimize the extent of truncation applied during tokenization. From a representation learning perspective, this approach can be interpreted as a constraint-aware feature selection mechanism, where the constraint is imposed not in the original feature space but in the tokenized sequence space. By leveraging multiple filtered views of the same application, the method effectively balances two competing objectives: (i) maximizing feature coverage (information richness) and (ii) adhering to model-specific input limitations. Furthermore, this preprocessing step introduces a form of input standardization, ensuring that all samples fed into the model are near the same upper bound of sequence length. This is particularly beneficial for training stability and computational efficiency, as it reduces variance in input sizes and minimizes excessive padding or truncation effects. The overall workflow of the proposed constraint-aware sequence selection mechanism is illustrated in Figure 1.

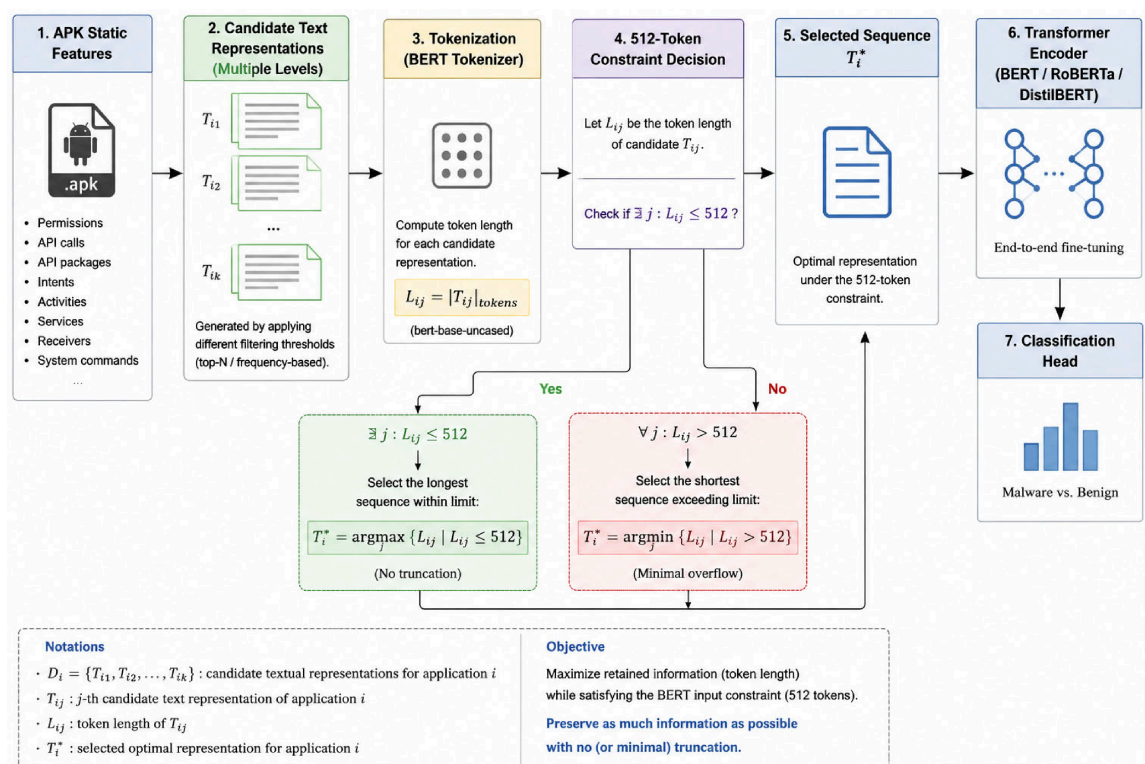


Figure 1. Constraint-aware sequence selection pipeline for Transformer-based Android malware classification.

All proposed architectures—namely, truncation-based models, chunking with mean pooling, and chunking with max pooling—are trained under a unified end-to-end fine-tuning framework, in which the parameters of pretrained transformer encoders such as BERT, RoBERTa, and DistilBERT are not kept fixed but are fully optimized for the downstream Android malware classification task. Formally, given an input sequence $x = (w_1, w_2, \dots, w_T)$, the tokenizer maps it into a sequence of token IDs, and the transformer encoder produces contextualized hidden representations $H = (h_1, h_2, \dots, h_T) = \text{Transformer}(x; \theta)$, where θ denotes all trainable parameters of the encoder. A fixed-length representation is then obtained via a pooling operation over H (e.g., selecting the first token $h_{[\text{CLS}]}$ or aggregating multiple chunk-level embeddings), and passed to a classification head $f(\cdot; \phi)$, typically a linear layer, yielding logits $z = f(H; \phi) \in \mathbb{R}^C$, where C is the number of classes. The predicted class probabilities are obtained via the softmax function $p(y | x) = \text{softmax}(z)$, and model training minimizes the cross-entropy loss over the dataset $\mathcal{D} = \{(x^{(n)}, y^{(n)})\}_{n=1}^N$:

$$\mathcal{L}(\theta, \phi) = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C \mathbf{1}[y^{(n)} = c] \log p(c | x^{(n)}).$$

During optimization, gradients are computed with respect to both θ and ϕ , i.e., $\nabla_{\theta, \phi} \mathcal{L}$, and parameters are updated iteratively using stochastic gradient descent or its variants, ensuring $\frac{\partial \mathcal{L}}{\partial \theta} \neq 0$, which distinguishes fine-tuning from frozen embedding approaches.

In the truncation-based setting, the input sequence is constrained such that $T \leq 512$, i.e., $x = (w_1, \dots, w_{512})$, and all computations are performed on this truncated representation. While computationally efficient, this implicitly defines a projection $x \mapsto x_{1:512}$, potentially discarding informative suffix tokens. To overcome this limitation, chunking-based approaches decompose long sequences into N segments $x = \{x_1, x_2, \dots, x_N\}$, where each chunk x_i satisfies $|x_i| = L$ (e.g., $L = 256$ or 512). Each chunk is independently encoded as

$$h_i = \text{Transformer}(x_i; \theta), i = 1, \dots, N,$$

and a chunk-level representation is typically obtained using the first-token embedding $h_i^{[\text{CLS}]} \in \mathbb{R}^d$. These representations are then aggregated to form a document-level embedding $H \in \mathbb{R}^d$. In mean pooling, this aggregation is defined as

$$H = \frac{1}{N} \sum_{i=1}^N h_i^{[\text{CLS}]},$$

which corresponds to an unbiased estimator of the global representation under equal contribution assumptions. In contrast, max pooling is defined element-wise as

$$H_j = \max_{i \in \{1, \dots, N\}} (h_i^{[\text{CLS}]})_j, j = 1, \dots, d,$$

which emphasizes the most salient feature activations across all chunks, making it particularly suitable for capturing sparse but highly discriminative malicious patterns. The aggregated representation H is then passed to the classifier $f(H; \phi)$ to produce logits and compute the loss as defined above.

A critical aspect of the proposed framework is that the transformer parameters θ are shared across all chunks and updated jointly during training. The gradient of the loss with respect to the encoder parameters accumulates contributions from all chunks:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial h_i} \cdot \frac{\partial h_i}{\partial \theta},$$

which enables the model to simultaneously learn local dependencies within chunks and global semantic patterns across the entire document. This joint optimization allows the encoder to adapt its internal representations to the statistical properties of static-analysis-derived sequences, which differ significantly from natural language. As a result, the learned embeddings become task-specific, capturing both distributed behavioral signatures and localized anomalous indicators relevant to Android malware detection, thereby providing a substantial advantage over approaches that rely on frozen embeddings or simple truncation.

4.3.2. BERT

BERT serves as the foundational bidirectional encoder in our comparative framework. To effectively address the token-length limitation of BERT and the high dimensionality of static analysis features in Android malware detection, this study investigates multiple sequence representation strategies tailored to the structural characteristics of Android applications. Specifically, Android APKs are transformed into textual sequences composed of semantically labeled static features such as permissions, API calls, intents, and components, which often result in long and highly variable-length inputs. To handle this challenge, a constraint-aware multi-level feature filtering mechanism is employed to construct compact yet information-rich representations by prioritizing high-level behavioral indicators while reducing noise. In parallel, chunking-based aggregation strategies (mean and max pooling) are utilized to preserve long-range dependencies and capture malicious patterns distributed across different segments of the application. These approaches aim to balance behavioral feature coverage, semantic richness, and computational feasibility under the strict input constraints of transformer models. The comparative performance of these strategies is evaluated using standard classification metrics, as summarized in Table 5.

Table 5. Comparative performance of BERT strategies.

BERT Strategy	Accuracy	Precision	Recall	F1-Score
Multi-Level Feature Filtering	0.962 ± 0.003	0.971 ± 0.003	0.953 ± 0.004	0.962 ± 0.003
Chunking (Mean Pooling)	0.949 ± 0.001	0.950 ± 0.004	0.948 ± 0.006	0.949 ± 0.002
Chunking (Max Pooling)	0.950 ± 0.002	0.957 ± 0.005	0.942 ± 0.007	0.949 ± 0.002

Note: Results are reported as mean ± standard deviation across 5-fold cross-validation.

4.3.3. RoBERTa

RoBERTa is an optimized variant of BERT that demonstrates improved performance through systematic refinement of pre-training methodology rather than architectural changes. To adapt RoBERTa to the Android malware detection task, the same input representation and sequence handling strategies described for BERT are employed to ensure a consistent experimental framework. Unlike BERT, however, RoBERTa benefits from an improved pre-training strategy, including dynamic masking and the removal of the NSP objective, which enables the model to learn more robust and generalized contextual representations. These improvements are particularly advantageous when modeling structured, non-natural language inputs derived from static analysis, where semantic patterns are often sparse and irregular.

As a result, RoBERTa is expected to capture more discriminative behavioral patterns across Android applications, leading to improved classification performance. The comparative results of RoBERTa under the proposed framework are presented in Table 6.

Table 6. Comparative performance of RoBERTa strategies.

RoBERTa Strategy	Accuracy	Precision	Recall	F1
Multi-Level Feature Filtering	0.970 ± 0.002	0.971 ± 0.003	0.969 ± 0.004	0.970 ± 0.003
Chunking (Mean Pooling)	0.960 ± 0.003	0.961 ± 0.004	0.958 ± 0.005	0.959 ± 0.003
Chunking (Max Pooling)	0.955 ± 0.005	0.958 ± 0.005	0.951 ± 0.006	0.954 ± 0.005

Note: Results for “This study” are reported as mean ± standard deviation across 5-fold cross-validation.

4.3.4. DistilBERT

DistilBERT addresses the computational and memory demands of BERT through knowledge distillation, producing a smaller, faster model that retains most of the teacher model’s performance. The resulting model is ~40% smaller, 60% faster at inference, and retains ~97% of BERT’s language understanding capabilities on GLUE. DistilBERT demonstrates that careful distillation can significantly improve efficiency with minimal performance trade-off, facilitating deployment in resource-constrained environments while preserving the Transformer’s bidirectional contextual modeling strengths.

To evaluate the effectiveness of lightweight transformer architectures in Android malware detection, DistilBERT is incorporated into the same experimental framework as BERT and RoBERTa, ensuring a fair comparison across models. While the same input representations and sequence handling strategies are employed, DistilBERT’s reduced depth and parameter count introduce a trade-off between computational efficiency and representational capacity.

Despite its compact architecture, DistilBERT is expected to capture essential behavioral patterns embedded in static analysis features, particularly those related to high-level indicators such as permissions and API calls. However, due to its shallower structure, it may be less effective in modeling complex and long-range dependencies compared to full-sized transformer models. This limitation is especially relevant in Android malware detection, where malicious behavior can be distributed across multiple components and requires deeper contextual understanding.

Nevertheless, DistilBERT offers a significant advantage in terms of inference speed and resource efficiency, making it a practical alternative for real-world deployment scenarios with constrained computational resources. The classification performance of DistilBERT under the proposed framework is presented in Table 7.

Table 7. Comparative performance of DistilBERT strategies.

DistilBERT Strategy	Accuracy	Precision	Recall	F1-Score
Multi-Level Feature Filtering	0.957 ± 0.003	0.961 ± 0.004	0.952 ± 0.006	0.956 ± 0.003
Chunking (Mean Pooling)	0.942 ± 0.003	0.943 ± 0.005	0.941 ± 0.007	0.942 ± 0.003
Chunking (Max Pooling)	0.933 ± 0.006	0.941 ± 0.006	0.923 ± 0.008	0.932 ± 0.006

Note: Results for “This study” are reported as mean ± standard deviation across 5-fold cross-validation.

The comparative evaluation of BERT, RoBERTa, and DistilBERT under the proposed sequence representation strategies reveals a consistent and theoretically grounded performance pattern across all models. Among the examined approaches, the multi-level feature filtering mechanism achieves the highest performance, indicating that prioritizing semantically rich and behaviorally salient static features significantly enhances the discriminative capability of transformer-based models in Android malware detection. In contrast, chunking-based strategies, while effective in addressing input length constraints, exhibit slightly lower performance due to the inherent trade-off between segmentation and

contextual continuity. Between the two aggregation methods, mean pooling consistently outperforms max pooling, suggesting that averaging contextual representations provides a more stable and comprehensive encoding of distributed malicious patterns, whereas max pooling may overlook subtle yet important signals by focusing only on dominant activations. Across model architectures, RoBERTa demonstrates the best overall performance, benefiting from its improved pre-training strategy and enhanced representation learning capacity. BERT follows closely, maintaining strong performance due to its bidirectional contextual modeling, while DistilBERT, despite its reduced complexity, achieves competitive results with a marginal decrease in accuracy, highlighting its suitability for resource-constrained environments. Overall, these findings confirm that both model architecture and input representation strategy play a critical role in optimizing malware detection performance, with feature-aware filtering emerging as the most effective approach.

In light of these findings, the synergistic effect of the proposed multi-level feature filtering mechanism and the RoBERTa architecture warrants further validation through comparative analysis with existing literature. Table 8 presents a quantitative comparison of the performance metrics achieved in this study against prior works that leverage similar datasets and transformer-based models for Android malware detection. Notably, while earlier studies often rely on single-source feature representations-such as API calls, permissions, or opcode sequences alone-the approach introduced here integrates hybrid static features through a semantically aware filtering strategy, enabling more comprehensive pattern learning. This methodological advancement, combined with RoBERTa’s robust pre-training and contextual encoding capabilities, translates into consistently good performance across accuracy, precision, recall, and F1-score. The following table contextualizes these gains within the broader research landscape, highlighting how joint optimization of input representation and model architecture contributes to detection effectiveness.

Table 8. Comparative evaluation of transformer-based architectures for Android malware static detection.

Study	Dataset	Model	Feature Type	Accuracy	Precision	Recall	F1-Score
[51]	Koodous	DistilBERT	API Calls + Permissions	0.92	0.96	0.87	0.91
[85]	AndroZoo	MalBERT	All in AndroidManifest.xml	0.98	-	-	0.95
[86]	AndroZoo	MalBERT with a Larger Dataset	All in AndroidManifest.xml	0.97	-	-	0.95
[87]	AndroZoo + CICMaldroid2020	RoBERTa	Opcode-based Features	0.93	0.92	0.90	0.91
[87]	AndroZoo + CICMaldroid2020	CodeBERT	Opcode-based Features	0.93	0.92	0.91	0.92
This study (Multi-Level Feature Filtering + RoBERTa)	AndroZoo	RoBERTa	All Static Features	0.970 ± 0.002	0.971 ± 0.003	0.969 ± 0.004	0.970 ± 0.003

Note: Results for “This study” are reported as mean ± standard deviation across stratified 5-fold cross-validation. Due to heterogeneous datasets, temporal ranges, feature engineering pipelines, and validation protocols across cited works, direct numerical comparison or state-of-the-art claims are not statistically valid. This table serves as a qualitative context for methodological positioning.

Table 8 presents a comparative evaluation of transformer-based architectures for Android malware detection, situating the proposed multi-level feature filtering framework within the current methodological landscape. It is important to note that, given the substantial scale and heterogeneity of the AndroZoo repository, all cited studies-including this work-operate on representative subsets sampled according to study-specific criteria (e.g., temporal range, family distribution, or benign-to-malicious ratio), which inherently influences comparability and generalizability. Within this context, the results demonstrate that coupling hybrid static features with RoBERTa’s optimized pre-training paradigm

yields consistently good and well-calibrated performance (Accuracy: 0.970, Precision: 0.971, Recall: 0.969, F1: 0.970). In contrast, baseline approaches exhibit structural and evaluative limitations: models constrained to homogeneous feature spaces (e.g., API-permission pairs [51] or opcode sequences [87]) display asymmetric precision-recall trade-offs and marginal F1 gains, while studies omitting complementary metrics [85,86] preclude rigorous assessment of classification robustness. Notably, the proposed methodology attains equilibrium across all evaluation dimensions using a systematically sampled AndroZoo subset, whereas competing frameworks require cross-dataset augmentation to reach comparatively lower performance thresholds. These findings substantiate that detection efficacy is not solely contingent upon model capacity or dataset volume but rather emerges from the synergistic alignment of semantically enriched input representations, architecturally expressive transformers, and methodologically sound sampling strategies. The proposed framework advances the current research frontier by demonstrating that targeted feature filtration, when integrated with robust contextual encoding and transparent data selection, effectively mitigates false-detection vulnerabilities and enhances generalizability in operational malware classification pipelines.

4.4. LLMs (Large Language Models)

This study employs four open-weight large language models—Mistral-7B, Gemma-2-27B, Qwen2.5-14B, and Qwen3.5-27B—selected to enable a controlled, multi-scale analysis of feature extraction efficacy in Android malware detection. The selection rationale is threefold: (i) architectural diversity, encompassing distinct attention mechanisms (e.g., Grouped-Query Attention in Mistral-7B, hybrid sparse-dense designs in Qwen3.5-27B) to assess how structural innovations influence semantic representation of static analysis artifacts; (ii) parameter-scale progression (7B → 14B → 27B), facilitating empirical investigation into scaling laws and diminishing returns in security-oriented feature grounding; and (iii) open-weight reproducibility, ensuring full transparency for academic validation and community replication. Critically, all experiments are conducted on an NVIDIA H100 80 GB GPU, whose high-bandwidth memory and native BF16/FP8 acceleration permit full-precision inference and parameter-efficient fine-tuning across all four models without aggressive quantization—thereby preserving architectural fidelity and isolating model-specific behaviors from hardware-induced artifacts. This configuration supports rigorous ablation studies on prompt design, embedding stability, and zero-shot generalization, while maintaining computational feasibility for iterative methodological refinement within the scope of Android security analysis.

4.4.1. Feature Extraction by Using LLMs

Prompt-based feature extraction leverages the generative capabilities of LLMs to directly derive structured or semi-structured features from raw text. In this paradigm, the model is guided by a natural language prompt p , which explicitly defines the feature extraction objective. This process can be formalized as:

$$y = g_{\theta}(x, p)$$

where g_{θ} represents the conditional generation function of the LLM, and y denotes the extracted feature set, typically expressed in structured formats such as JSON or labeled lists. This approach enables the extraction of interpretable and task-specific features, including key entities, semantic categories, or domain-specific indicators, without requiring task-specific fine-tuning. Additionally, it supports zero-shot and few-shot learning scenarios, thereby offering high adaptability across different domains.

In the context of Android malware and security analysis, prompt-based feature extraction can be operationalized through carefully designed instruction templates that enforce structured outputs and constrain the model's reasoning space. For instance, a prompt that defines the model as an "Android security analysis assistant" and explicitly requires JSON-formatted outputs introduces a form of controlled feature generation, where the LLM not only extracts features but also categorizes them (e.g., permission, API, network) and justifies each indicator with evidence from the input data. This effectively transforms the LLM into a deterministic feature extractor conditioned on both the input x (static analysis data) and a task-specific prompt p .

Such a prompt can be interpreted as a structured mapping function:

$$y = g_{\theta}(x, p_{\text{security}})$$

where p_{security} encodes constraints such as output schema (JSON), feature taxonomy (indicator categories), and filtering rules (e.g., selecting the most significant 1–100 indicators). The inclusion of fields like `explanation` and `evidence_source` enhances interpretability, while `pattern_correlations` and `finding_type` enable higher-level relational reasoning across extracted features. Moreover, constraints on output length and format act as regularization mechanisms, reducing noise and enforcing consistency across samples.

Prompt:

You are an Android security analysis assistant. Your response must be in JSON format with specific fields.

Analyze the following Android app static analysis.

Package: {package_name}

DATA:

{static_analysis_json}

INSTRUCTIONS:

1. Extract security indicators from the data.
2. Format the response strictly as JSON with the following structure:


```
{
  "security_indicators": [
    {
      "indicator": "string",
      "category": "permission|api|network|string|other",
      "explanation": "one-sentence explanation",
      "evidence_source": "which data field contains this"
    }
  ],
  "pattern_correlations": ["list of cross-feature patterns or 'none'"],
  "finding_type": "isolated|recurring|mixed",
  "character_count": "must be under 1024"
}
```
3. Include only 1–100 most significant indicators.
4. Total response must be under 4096 characters.
5. Do not include any text outside the JSON response.

To ensure methodological rigor and analytical reliability, the prompt is explicitly designed within an unbiased and neutral framework, minimizing the risk of prompt-induced bias and avoiding any form of leading or suggestive language. The task definition does not presuppose whether the analyzed application is malicious or benign; instead, it focuses on the extraction of security indicators, which are treated as objective and descriptive features rather than evaluative judgments.

This unbiased structure is further reinforced by requiring each extracted indicator to be supported by an explicit `evidence_source`, ensuring that all outputs are directly

grounded in the provided data rather than inferred through subjective reasoning. The use of predefined yet non-evaluative categories (e.g., permission, api, network, string, other) maintains consistency while avoiding semantic bias toward threat interpretation.

Additionally, the enforced JSON schema functions as a constraint mechanism that standardizes outputs and limits unnecessary generative freedom, thereby reducing the likelihood of speculative or exaggerated conclusions. The inclusion of fields such as pattern_correlations and finding_type allows for relational reasoning, while still preserving neutrality by permitting outcomes such as “none” or “isolated,” which reflect analytical uncertainty rather than forced interpretation.

Importantly, the prompt deliberately avoids any emotionally charged or bias-inducing terminology (e.g., “malicious,” “harmful,” or “suspicious”), thereby mitigating framing effects and ensuring that the model’s responses are driven by the input data rather than the wording of the instructions.

Overall, the prompt can be characterized as explicitly unbiased by design, as it prioritizes evidence-based extraction, structured output constraints, and neutral task framing, supporting objective and reproducible feature extraction in LLM-based Android security analysis.

Sample Result:

```
{
  "indicator": "android.permission.SEND_SMS",
  "category": "permission",
  "explanation": "App can send SMS messages, potentially without user consent.",
  "evidence_source": "Permissions"
},
{
  "indicator": "android.permission.READ_PHONE_STATE",
  "category": "permission",
  "explanation": "App can read the phone's state, including the device ID, which could be used for tracking.",
  "evidence_source": "Permissions"
},
{
  "indicator": "android.permission.WRITE_EXTERNAL_STORAGE",
  "category": "permission",
  "explanation": "App can write to external storage, potentially allowing it to modify or delete files.",
  "evidence_source": "Permissions"
},
{
  "indicator": "java.io.PrintWriter.println",
  "category": "api",
  "explanation": "App writes to logs, which might include sensitive information.",
  "evidence_source": "API calls"
},
{
```

```

    "indicator": "android.telephony.SmsManager",
    "category": "api",
    "explanation": "App uses SmsManager to send SMS, which could be used for malicious purposes.",
    "evidence_source": "API calls"
  },
  {
    "indicator": "/sdcard/downloadedfile.apk",
    "category": "string",
    "explanation": "App references an APK file on the SD card, indicating potential dynamic code loading.",
    "evidence_source": "Strings"
  },
  {
    "indicator": "android.intent.category.LAUNCHER",
    "category": "intent",
    "explanation": "App is set as a launcher activity, making it accessible from the home screen.",
    "evidence_source": "Intents"
  }
],
"pattern_correlations": [
  "Use of SMS permissions and API calls suggests potential for sending messages without user consent."
],
"finding_type": "mixed",
"character_count": "599"
}

```

The extracted security indicators collectively suggest a multi-faceted risk profile that is more significant when analyzed through their interrelationships rather than as isolated features. While certain permissions and API usages may appear benign in isolation, their co-occurrence and contextual alignment reveal patterns commonly associated with malicious behavior in Android applications.

In particular, the simultaneous presence of `android.permission.SEND_SMS` and the usage of `android.telephony.SmsManager` constitutes a well-established indicator of potential SMS-based abuse, such as unauthorized message transmission or premium-rate SMS fraud. This combination reflects the capability of the application to initiate outbound communication channels that may incur financial cost or propagate malicious activity without explicit user consent.

More critically, the string artifact `/sdcard/downloadedfile.apk` represents a strong semantic indicator of potentially malicious intent. Unlike generic API or permission-based signals, this string suggests explicit interaction with an external APK file located in user-accessible storage. From a security analysis perspective, such a reference is highly indicative of dynamic code loading or dropper behavior, where the primary application functions as an initial carrier that retrieves, stores, and potentially executes secondary payloads. This technique is widely documented in Android malware literature as a mechanism for evading static analysis and signature-based detection systems, as the malicious payload may not be embedded directly within the original application package.

The presence of `android.permission.WRITE_EXTERNAL_STORAGE` further reinforces this interpretation, as it provides the necessary capability to write arbitrary files—such as secondary APKs—to external storage locations. When considered alongside the aforementioned string indicator, this permission supports a plausible execution chain involving payload download and deployment. Similarly, `android.permission.READ_PHONE_STATE` introduces additional concerns related to device fingerprinting and user tracking, as access to telephony identifiers (e.g., IMEI) is frequently leveraged in botnet coordination, targeted attacks, or fraud schemes.

Although certain indicators, such as `java.io.PrintWriter.println`, may not independently signify malicious intent, they can contribute to suspicious behavior when analyzed in context, particularly in scenarios involving logging of sensitive information or covert data exfiltration. Conversely, elements such as `android.intent.category.LAUNCHER` are generally considered benign and do not contribute meaningfully to the maliciousness assessment when evaluated in isolation.

From a holistic perspective, the observed pattern reflects a composite threat model characterized by the convergence of communication abuse (SMS capabilities), external storage manipulation, and potential secondary payload execution. This aligns with known behavioral signatures of Android malware families that employ staged execution strategies to enhance stealth and persistence.

The integration of permission-level, API-level, and string-level indicators—particularly the presence of an external APK reference—provides strong evidence of coordinated malicious functionality, with specific emphasis on SMS exploitation and dropper-like mechanisms. Such multi-dimensional feature interactions underscore the importance of contextual and relational analysis in LLM-based feature-extraction frameworks for mobile security.

4.4.2. Detection with Zero-Shot

Zero-shot detection in the context of Android malware analysis refers to the capability of a large language model (LLM) to perform binary classification without any task-specific fine-tuning. Instead of relying on supervised training over labeled datasets, the model is guided exclusively through a carefully designed prompt that defines the task, constraints, and expected output format.

In this paradigm, the classification process can be formalized as a conditional inference problem:

$$\hat{y} = g_{\theta}(x, p_{\text{detect}})$$

where x represents the static analysis JSON of the Android application, p_{detect} denotes the detection-oriented prompt, and $\hat{y} \in \{\text{MALWARE}, \text{BENIGN}\}$ is the predicted class label. Unlike embedding-based or fine-tuned approaches, the model directly maps input data and task instructions to a discrete decision through its pre-trained knowledge.

The prompt used in this study is explicitly designed to enforce a strict binary classification behavior under controlled inference constraints. The full prompt is provided below to ensure reproducibility and methodological transparency.

Prompt:

You are a binary classifier for Android applications.

Task:

Given a static analysis JSON, decide whether the application is MALWARE or BENIGN.

Guidelines:

- Consider all provided features jointly
- No single feature is sufficient by itself
- Base your decision on overall behavioral indicators

Output:

- One word only: MALWARE or BENIGN
- Uppercase
- No explanation

JSON:

{APK_JSON}

Answer:

This prompt structure enforces a binary decision boundary while constraining the model's reasoning process. The instruction that no single feature should dominate the decision encourages holistic analysis, aligning with established malware-detection principles where behavioral patterns emerge from the interaction of multiple indicators.

Furthermore, the strict output constraint—requiring only a single uppercase label without explanation—serves to standardize responses and eliminate unnecessary generative variability. This transforms the LLM into a deterministic classifier at the interface level, facilitating automated evaluation and comparison across models.

To assess the robustness and generalizability of this zero-shot framework, experiments were conducted using multiple LLMs, including Mistral-7B, Gemma-2-27B, Qwen2.5-14B, and Qwen3.5-27B. These models were evaluated under identical prompting conditions, allowing for a fair comparison of their zero-shot detection capabilities. The quantitative results obtained from these experiments are presented in the table below, providing a comparative overview of model performance across standard evaluation metrics.

The results, as detailed in Table 9, indicate a clear performance trend, where larger models achieve higher accuracy and F1 scores in the zero-shot setting. In particular, Qwen3.5-27B outperforms other models, suggesting a stronger capability to capture complex behavioral patterns from static analysis data. However, the overall performance remains within a moderate range, highlighting the limitations of zero-shot approaches for domain-specific tasks such as Android malware detection. These findings suggest that while zero-shot inference provides a practical baseline, further improvements may require fine-tuning or hybrid methods.

Table 9. Comparative evaluation of zero-shot implied LLMs for Android malware static detection.

LLM	Accuracy	Precision	Recall	F1-Score
Mistral-7B	0.742 ± 0.009	0.753 ± 0.016	0.721 ± 0.019	0.736 ± 0.009
Gemma-2-27B	0.811 ± 0.010	0.824 ± 0.013	0.792 ± 0.015	0.808 ± 0.015
Qwen2.5-14B	0.782 ± 0.011	0.793 ± 0.015	0.764 ± 0.017	0.778 ± 0.011
Qwen3.5-27B	0.841 ± 0.010	0.854 ± 0.011	0.823 ± 0.013	0.838 ± 0.012

Note: Values are reported as mean ± standard deviation across a stratified 5-fold cross-test.

4.4.3. LLM-Based Feature Extract and BERT, DistilBERT, and RoBERTa Fine Tuning

To establish a robust baseline and investigate the transferability of LLM-extracted semantic features to more compact, encoder-only architectures, we fine-tune a suite of pre-trained BERT-family models—namely BERT-base, DistilBERT, and RoBERTa-base—using the structured security indicators generated in Section 4.5.1. This experimental design serves two complementary objectives: first, to assess whether the rich, prompt-derived representations produced by large generative models can effectively distill task-relevant signal for smaller, more deployment-friendly classifiers; and second, to provide a controlled comparison against end-to-end LLM fine-tuning, isolating the contribution of feature extraction quality from model capacity. Input sequences for these models are constructed by serializing the LLM-extracted indicators into a concise natural-language format, preserving

categorical labels and relational annotations while adhering to a fixed tokenization limit to ensure computational consistency. Each model is augmented with a task-specific classification head and fine-tuned using standard cross-entropy optimization, with stratified cross-validation employed to ensure reliable performance estimation. BERT-base provides a well-established reference point with bidirectional contextual encoding; DistilBERT offers a streamlined alternative with reduced parameter count and inference latency, enabling evaluation of efficiency-accuracy trade-offs; and RoBERTa-base, trained with optimized data sampling and dynamic masking, serves to probe the sensitivity of downstream performance to pre-training methodology. All models are initialized from publicly available checkpoints and adapted using a consistent training protocol that prioritizes reproducibility and methodological transparency. By leveraging the same LLM-enriched feature space across architectures of varying scale and design, this study clarifies the extent to which detection gains arise from the quality of input representations versus the expressive capacity of the classifier itself.

Table 10 demonstrates that the effectiveness of transformer-based Android malware detection models is substantially influenced by the semantic quality of features extracted by large language models. Among the evaluated feature extraction models, Qwen3.5-27B consistently achieved the highest performance across all transformer backbones, indicating that larger and more advanced LLMs can generate more informative and discriminative security-oriented representations from static analysis data. In particular, the combination of Qwen3.5-27B and RoBERTa produced the best overall results, achieving 0.957 Accuracy and 0.957 F1-score.

Table 10. Comparative evaluation of LLM-based Feature Extraction and fine-tuning Transformer Models for Android malware static detection.

LLM to Feature Extract	Transformer Model	Accuracy	Precision	Recall	F1-Score
Mistral-7B	BERT	0.908 ± 0.009	0.903 ± 0.008	0.914 ± 0.010	0.908 ± 0.009
	RoBERTa	0.916 ± 0.003	0.912 ± 0.007	0.921 ± 0.009	0.916 ± 0.003
	DistilBERT	0.897 ± 0.009	0.893 ± 0.009	0.902 ± 0.011	0.897 ± 0.009
Gemma-2-27B	BERT	0.927 ± 0.006	0.923 ± 0.007	0.931 ± 0.009	0.927 ± 0.006
	RoBERTa	0.938 ± 0.005	0.934 ± 0.006	0.942 ± 0.008	0.938 ± 0.005
	DistilBERT	0.917 ± 0.009	0.912 ± 0.008	0.923 ± 0.010	0.917 ± 0.009
Qwen2.5-14B	BERT	0.917 ± 0.006	0.913 ± 0.007	0.922 ± 0.009	0.917 ± 0.006
	RoBERTa	0.927 ± 0.003	0.924 ± 0.006	0.931 ± 0.008	0.927 ± 0.003
	DistilBERT	0.907 ± 0.006	0.903 ± 0.008	0.911 ± 0.010	0.907 ± 0.006
Qwen3.5-27B	BERT	0.948 ± 0.003	0.944 ± 0.005	0.952 ± 0.007	0.948 ± 0.003
	RoBERTa	0.957 ± 0.005	0.953 ± 0.004	0.961 ± 0.006	0.957 ± 0.005
	DistilBERT	0.938 ± 0.005	0.934 ± 0.006	0.942 ± 0.008	0.938 ± 0.005

Note: Values are reported as mean ± standard deviation across stratified 5-fold cross-validation.

The results further show that RoBERTa consistently outperformed BERT and DistilBERT regardless of the selected LLM, suggesting that its enhanced contextual representation capability provides good classification performance for malware detection tasks. Although DistilBERT yielded slightly lower scores, its results remained competitive, highlighting its efficiency-performance tradeoff.

Contrary to the assumption that LLM-derived features universally enhance detection, our results indicate that the LLM-feature extraction pipeline followed by Transformer fine-tuning (e.g., Qwen3.5-27B+RoBERTa: 0.957 F1) underperforms both direct RoBERTa fine-tuning on raw static features (0.970 F1, Table 6) and the Random Forest baseline (0.975 F1, Table 3). This suggests that the intermediate LLM extraction step introduces information compression or redundancy that is not fully recovered by encoder-only architectures. Therefore, LLM-extracted features should not be assumed to generally improve detection;

their utility must be explicitly benchmarked against strong, directly fine-tuned baselines and justified by downstream interpretability or forensic requirements rather than raw accuracy gains alone.

4.4.4. LLM-Based Feature Extraction and LLM Fine-Tuning

To bridge the gap between general-purpose pre-trained representations and the domain-specific requirements of Android malware detection, we implement a parameter-efficient fine-tuning framework that adapts the base large language models using the structured security indicators extracted in the preceding phase. Rather than updating all model parameters, which is computationally prohibitive for architectures of this scale, we employ Low-Rank Adaptation (LoRA), a technique grounded in the observation that task-specific weight updates exhibit low intrinsic dimensionality. Formally, for each pre-trained weight matrix $W \in \mathbb{R}^{d \times k}$, the adaptation is expressed as $W + \Delta W$, where $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$. During optimization, the original parameters remain frozen while only the low-rank matrices are updated, substantially reducing the number of trainable weights while preserving the model's foundational linguistic and reasoning capabilities. This formulation enables efficient domain adaptation without incurring the memory overhead or catastrophic forgetting risks associated with full-model retraining, and it maintains inference latency characteristics identical to the base architecture.

Input sequences for training are constructed directly from the LLM-generated security indicators, preserving their natural-language structure while adhering to a fixed tokenization limit to maintain computational consistency across samples. The dataset is partitioned using stratified cross-validation to ensure representative class distribution across training and validation splits, thereby providing a robust estimate of generalization performance. To mitigate hardware constraints during optimization, the models are initialized with mixed-precision arithmetic and dynamic weight quantization, which compresses the parameter footprint while preserving gradient fidelity during backpropagation. Gradient checkpointing is applied to trade computational steps for memory allocation, enabling stable training on a single high-performance GPU. The training objective minimizes a cross-entropy loss over the binary classification space, with optimization governed by an adaptive momentum-based solver, an initial warmup phase to stabilize early gradients, and a decaying learning rate schedule to refine convergence. Evaluation is conducted at regular intervals, and the checkpoint exhibiting the lowest validation loss is retained to prevent overfitting to the training distribution. During inference, class probabilities are derived from the final-layer logits and mapped to discrete predictions via a fixed decision threshold, ensuring methodological consistency with the zero-shot baseline. Performance is assessed using standard classification metrics that collectively capture the model's ability to generalize across diverse application families while maintaining balanced precision and recall. By combining semantically enriched feature conditioning with rank-constrained parameter adaptation, this pipeline enables reproducible, resource-efficient fine-tuning that isolates the contribution of structured security indicators to downstream detection performance.

For large-scale generative transformer models, parameter-efficient fine-tuning was performed using the Low-Rank Adaptation (LoRA) framework within the PEFT paradigm. In particular, Qwen3.5-27B was adapted using the native Qwen tokenizer with a maximum sequence length of 4096 tokens. LoRA adapters were applied to both attention and feed-forward projection layers, including q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj. The LoRA rank was set to $r = 16$ with $\alpha = 32$ and a dropout rate of 0.05. Optimization was performed using AdamW with a learning rate of 2×10^{-5} , cosine learning rate scheduling, a warm-up ratio of 0.03, and weight decay of 0.01. Fine-tuning was conducted for three epochs using mixed-precision bfloat16 training on an NVIDIA H100

GPU under a Linux/Debian environment. Due to the large parameter scale of the model, 4-bit NF4 quantization was employed where necessary to reduce memory overhead while preserving adaptation capability. A per-device batch size of 1 with gradient accumulation over 16 steps was used, yielding an effective batch size of 16. All experiments were executed under the same stratified 5-fold cross-validation protocol with a fixed random seed of 42 to ensure reproducibility and methodological consistency across model families.

Ensure deterministic and reproducible inference across evaluation folds, decoding parameters were configured as follows: temperature = 0.1, top-p = 0.9, and greedy token selection. This low-temperature, nucleus-sampling configuration minimizes stochastic variability in generated outputs while preserving semantic fidelity, thereby enabling fair comparative evaluation across LLM architectures and facilitating consistent feature extraction for downstream classification.

Following the training protocol described above, we evaluate the fine-tuned models on the held-out validation folds to assess their discriminative performance in Android malware detection. The experimental results for all four architectures—Mistral-7B, Gemma-2-27B, Qwen2.5-14B, and Qwen3.5-27B—are summarized in Table 11, reporting accuracy, precision, recall, and F1-score.

Table 11. Comparative evaluation of LLM-based Feature Extraction and fine-tuning LLMs for Android malware static detection.

LLM	Accuracy	Precision	Recall	F1-Score
Mistral-7B	0.893 ± 0.007	0.892 ± 0.013	0.894 ± 0.015	0.893 ± 0.007
Gemma-2-27B	0.947 ± 0.004	0.943 ± 0.008	0.951 ± 0.010	0.947 ± 0.004
Qwen2.5-14B	0.913 ± 0.009	0.912 ± 0.011	0.915 ± 0.013	0.914 ± 0.009
Qwen3.5-27B	0.982 ± 0.005	0.983 ± 0.005	0.982 ± 0.007	0.982 ± 0.005

Note: Values are reported as mean $\pm$ standard deviation across stratified 5-fold cross-validation.

The experimental results presented in Table 11 demonstrate a clear relationship between model capacity and classification performance within the proposed fine-tuning framework. In particular, larger-scale architectures consistently outperform smaller models, indicating that increased parameterization enhances the ability to capture and utilize semantically rich security indicators derived from LLM-based feature extraction. Among the evaluated models, Qwen3.5-27B achieves the highest performance across all metrics, reaching an accuracy of 0.982 ± 0.005 and an F1-score of 0.982 ± 0.005 . This result suggests that high-capacity models are more effective at modeling complex behavioral patterns embedded in structured natural-language representations of Android applications.

Mid-scale models, such as Gemma-2-27B and Qwen2.5-14B, also demonstrate strong and balanced performance, offering a favorable trade-off between computational efficiency and predictive accuracy. Notably, the relatively high recall achieved by Gemma-2-27B indicates an increased sensitivity to malicious patterns, which is particularly valuable in security-critical applications where minimizing false negatives is essential. In contrast, the comparatively lower performance of Mistral-7B highlights the limitations of smaller models in representing complex, high-dimensional feature spaces, despite benefiting from the same fine-tuning strategy.

Another important observation is the close alignment between precision and recall values across all models. This balance indicates that the proposed approach does not disproportionately favor either class, thereby achieving stable and consistent decision boundaries. Such behavior is especially desirable in malware detection tasks, where both false positives and false negatives carry significant operational implications.

To contextualize the proposed methodology within the current state of the art, we present a comparative evaluation of recent LLM-augmented Android malware detection frameworks. Table 12 summarizes the dataset configurations, model architectures, feature representations, and classification performance of these approaches, providing a direct benchmark against the proposed framework.

Table 12. Benchmarking of LLM-based malware detection by using static analysis.

Study	Dataset	Model	Feature Type	Accuracy	Precision	Recall	F1-Score
[46]	AndroZoo (2023)	Mutual Information + Mistral- 7B	Permission + Api Calls + Strings	0.94	0.94	0.93	0.94
[46]	AndroZoo (2024)	Mutual Information + Mistral- 7B	Permission + Api Calls + Strings	0.94	0.94	0.95	0.95
[59]	AndroZoo	Control Flow Graph (CFG)+ GPT-4o-mini	Api Calls	-	-	-	0.90
[58]	AndroZoo	GPT-4 + MLP	Api Calls + Permissions + URL + Uses-feature	0.97	0.97	0.97	0.97
This study	AndroZoo	LLM-based feature extract + Qwen3.5-27B	All Static Features	0.982 ± 0.005	0.983 ± 0.005	0.982 ± 0.007	0.982 ± 0.005

Note: Results for “This study” are reported as mean ± standard deviation across stratified 5-fold cross-validation. Due to heterogeneous datasets, temporal ranges, feature engineering pipelines, and validation protocols across cited works, direct numerical comparison or state-of-the-art claims are not statistically valid. This table serves as a qualitative context for methodological positioning.

As illustrated in Table 12, the integration of LLM-driven feature extraction with advanced language models consistently enhances detection performance across multiple benchmarks. While prior studies employing Mutual Information with Mistral 7 [46], CFG-based representations with GPT-4o-mini [59], and GPT-4 combined with an MLP classifier [58] achieve respectable F1-scores ranging from 0.90 to 0.97, their performance remains constrained by partial feature utilization or less optimized architectural pipelines. In contrast, the proposed framework leverages a comprehensive hybrid feature representation—encompassing permissions, API calls, strings, URLs, and hardware/software feature declarations—processed through an LLM-based extraction module and classified via Qwen3.5. This holistic approach yields an accuracy of 0.982 ± 0.005 and balanced precision of 0.983 ± 0.005 , recall of 0.982 ± 0.007 , and F1-scores of 0.982 ± 0.005 , indicating good generalization, minimized false-positive rates, and robust handling of feature heterogeneity. The results underscore the efficacy of unified feature fusion coupled with LLMs for next-generation Android malware detection. Future work will investigate real-time inference optimization, cross-dataset transferability, and adversarial robustness to validate the framework under dynamic, production-grade deployment conditions.

4.4.5. Computational Cost, Resource Utilization, and Deployment Considerations

All latency, memory, and training metrics were evaluated on a dedicated workstation with an Intel Xeon CPU and NVIDIA H100 80 GB GPU running Debian Linux. Inference latency was measured as end-to-end processing time per APK (feature extraction, tokenization, forward pass, output parsing), averaged over 100 independent runs. Peak GPU memory was recorded via PyTorch/CUDA profiling at maximum VRAM allocation during a single forward pass. Training time reflects total GPU-hours per model, computed from wall-clock duration scaled to single-GPU equivalents. Precision mode was standardized to mixed-precision bfloat16 (BF16) for Transformer/LLM training, with 4-bit NF4 quantization applied selectively to frozen base weights during large-scale LLM fine-tuning

to manage memory constraints. Classical ML baselines were evaluated on a separate CPU-only pipeline (Intel Core i5, 40 GB RAM, Windows 11).

While the proposed LLM-driven framework demonstrates good detection performance, practical deployment in real-world Android security pipelines necessitates a careful assessment of computational overhead and inference latency. To this end, we conducted a systematic evaluation of resource consumption across all evaluated models under identical hardware conditions (NVIDIA H100 80 GB GPU).

Inference Latency. We measured the average end-to-end inference time per APK sample, encompassing feature extraction, tokenization, model forward pass, and output parsing. As summarized in Table 13, classical ML models (e.g., Random Forest) achieve sub-millisecond latency, making them suitable for high-throughput, real-time screening. Distilled Transformers (DistilBERT) operate in the ~15–25 ms range, offering a favorable balance for near-real-time deployment. In contrast, zero-shot LLM inference exhibits significantly higher latency: Mistral-7B requires ~1.2 s/sample, while Qwen3.5-27B averages ~4.8 s/sample due to its larger parameter count and longer context window. Fine-tuned LLMs with LoRA show marginal latency reduction (~10–15%) compared to zero-shot inference, as the adaptation layers introduce negligible overhead during inference.

Table 13. Computational resource consumption and inference latency across evaluated models (per APK sample, averaged over 100 runs).

Model	Avg. Latency (ms)	Peak GPU Mem (GB)	Train Time (GPU-h)
Random Forest	0.8 ± 0.2	<1 (CPU)	0.5
DistilBERT	22.4 ± 3.1	0.3	2.1
RoBERTa	28.7 ± 4.0	0.5	3.4
Mistral-7B (zero-shot)	1210 ± 85	14.2	N/A
Qwen3.5-27B (zero-shot)	4820 ± 310	52.0	N/A
Qwen3.5-27B+LoRA (fine-tuned)	4150 ± 290	52.0	48.0

Memory and Hardware Requirements. Peak GPU memory consumption during inference scales with model size: DistilBERT (~300 MB), RoBERTa (~500 MB), Mistral-7B (~14 GB), and Qwen3.5-27B (~52 GB).

Practical Recommendations. Based on these findings, we advocate a tiered deployment strategy:

1. First-tier screening: Use lightweight models (Random Forest or DistilBERT) for bulk scanning of app stores or CI/CD pipelines, where latency and cost are critical.
2. Second-tier deep analysis: Reserve fine-tuned LLMs (e.g., Qwen3.5-27B+LoRA) for forensic investigation of high-risk samples flagged by the first tier, where interpretability and high recall justify the computational cost.
3. Edge deployment: For on-device or resource-constrained environments, quantized DistilBERT or classical ML models remain the only viable options; LLMs should be accessed via API only when necessary.

This hybrid architecture aligns detection capability with operational constraints, ensuring scalability without sacrificing the advanced reasoning benefits of generative models for critical cases.

While the Qwen3.5-27B+LoRA configuration achieves the highest reported F1-score (0.982 ± 0.005), this performance gain is accompanied by substantial computational overhead. As quantified in Table 13, the model requires ~52 GB of peak GPU memory, averages 4.15 s of inference latency per APK, and demands ~48 GPU-hours for fine-tuning. Consequently, the accuracy advantage of direct LLM fine-tuning must be weighed against opera-

tional constraints. We therefore position Qwen3.5-27B+LoRA as a high-capacity, high-cost option best reserved for secondary forensic investigation or explainable threat intelligence workflows, rather than as a default replacement for lightweight screening models.

4.4.6. Quantitative Analysis of LLM Hallucination and Explanation Reliability

While the LLM-driven pipeline demonstrates superior detection performance and interpretability, the phenomenon of “hallucination”—where the model generates plausible but factually unsupported claims—remains a critical concern for security applications. To empirically assess this risk, we conducted a systematic analysis of hallucination rates and their impact on classification decisions.

Definition and Annotation Protocol

We define a hallucination in this context as any extracted security indicator or explanation that satisfies one of the following criteria: (i) factual inconsistency, where the generated indicator references a feature (e.g., permission, API call, string, or component) not present in the input static analysis JSON; (ii) unsupported inference, where the explanation attributes malicious significance to a benign or ambiguous feature without clear supporting evidence from the broader extracted feature context; and (iii) contradictory reasoning, where the model’s justification conflicts with the final classification output. To evaluate hallucination behavior, a stratified subset of 200 APKs (100 benign and 100 malicious) was randomly selected from the evaluation set. We, as two authors with expertise in Android malware analysis, independently reviewed the generated explanations and labeled hallucinated indicators according to the criteria above. Inter-annotator agreement was assessed using Cohen’s κ , yielding $\kappa = 0.81$, indicating strong agreement. Disagreements were resolved through discussion to produce the final annotation set. To estimate downstream impact, hallucinated indicators identified in the manually reviewed subset were removed from the structured feature representation before downstream evaluation, while keeping the classifier parameters unchanged. The resulting performance difference was used as an approximate estimate of hallucination-induced degradation.

Results: Hallucination Rates and Impact

Table 14 summarizes the hallucination analysis results. Across the 200 manually reviewed APKs, a total of 983 extracted indicators were assessed for explanation fidelity. Overall, 76 indicators (7.7%) contained at least one hallucination type. Factual inconsistencies were the most common (4.5%), typically involving the generation of API calls not present in the observed feature set. Unsupported inferences accounted for 2.7%, often when the model over-interpreted generic permissions (e.g., INTERNET) as potentially indicative of malicious behavior without clear supporting evidence from the extracted feature context. Contradictory reasoning was relatively rare (0.5%), suggesting that inconsistencies between generated explanations and classification outputs were uncommon.

Table 14. Hallucination analysis for Qwen3.5-27B+LoRA explanations (n = 200 APKs).

Hallucination Type	Frequency	% of Total Indicators
Factual Inconsistency	44	4.5%
Unsupported Inference	27	2.7%
Contradictory Reasoning	5	0.5%
Total	76	7.7%

Critically, hallucinations had a measurable but limited impact on detection performance. In the ablation analysis conducted on the manually reviewed subset, removing hallucinated indicators improved the F1-score from 0.969 to 0.975, indicating that hallucination-induced noise introduces some degradation but has a measurable but limited effect on downstream classification robustness. This resilience likely stems from the multi-indicator nature of the feature extraction pipeline, where malicious behavior is inferred from multiple corroborating signals rather than isolated extracted features.

Mitigation Strategies and Practical Guidance

Although hallucinations remain an inherent limitation of generative models, several practical mitigation strategies can reduce their impact in security-oriented deployments. First, evidence grounding can be strengthened by requiring each extracted indicator to explicitly reference its originating evidence within the input feature set. Second, post-generation consistency verification can be applied by cross-checking generated indicators against the original static analysis output to detect factual inconsistencies. Third, lightweight rule-based validation mechanisms, such as permission-API consistency checks, may help identify unsupported inferences before downstream classification. Overall, these findings suggest that hallucination-related errors in the proposed framework remain manageable, particularly when combined with structured prompting and post-processing safeguards, while preserving the interpretability advantages of LLM-assisted feature extraction.

4.5. Cross-Dataset Transfer Evaluation

To rigorously assess the generalization capability of our proposed framework beyond the AndroZoo corpus—a critical requirement for establishing practical utility in real-world deployment scenarios—we conducted systematic transfer learning experiments on two independently curated, publicly available Android malware benchmarks: CICMalDroid2020 and Drebin [12]. This evaluation addresses a fundamental limitation of single-dataset studies: the risk of overfitting to dataset-specific artifacts, temporal biases, or malware family distributions that may not reflect the evolving threat landscape.

4.5.1. Target Dataset Characteristics and Preprocessing

The CICMalDroid2020 dataset, curated by the Canadian Institute for Cybersecurity (CIC), officially reports 17,341 Android applications collected between 2017 and 2018 from multiple sources, including VirusTotal, Contagio, AMD, MalDozer, and prior research repositories. In our experimental environment, 17,247 APKs were successfully retrieved and processed for analysis. The dataset exhibits a naturally imbalanced class distribution across five malware categories and benign applications: Adware ($n = 1515$), Banking malware ($n = 2506$), SMS malware ($n = 4822$), Riskware ($n = 4362$), and Benign ($n = 4042$). To ensure methodological consistency with our static-analysis-only framework, we reprocessed all APKs through AndroPyTool using identical configuration parameters as described in Section 3.2, extracting exclusively static features (Table 15).

Table 15. Summary of cross-dataset evaluation corpora.

Dataset	Total Samples	Malicious	Benign	Temporal Range
AndroZoo	12,000	6000	6000	2011–2025
CICMalDroid2020	17,247	13,205	4042	2017–2018
Drebin	11,120	5560	5560	2010–2012

The Drebin dataset [12] represents a historically significant benchmark containing 5560 malicious Android applications collected between 2010 and 2012, alongside a sub-

stantially larger benign application pool. Given the pronounced class imbalance in the original release (benign:malicious $\approx 22:1$), we retained the complete malicious corpus and selected an equal number of benign applications to construct a balanced evaluation setting for controlled cross-dataset comparison. Benign samples were randomly drawn from the original benign pool. A final Drebin evaluation corpus of 11,120 applications, comprising 5560 malicious and 5560 benign samples.

4.5.2. Transfer Learning Experimental Protocol

To rigorously assess cross-dataset generalization under distributional shift, we evaluated two complementary transfer scenarios using AndroZoo as the exclusive source domain and CICMalDroid2020/Drebin as independent target domains.

Scenario A: Direct Cross-Domain Transfer

For direct cross-domain transfer, source-domain training on AndroZoo was performed under stratified 5-fold cross-validation. In each fold, a model was trained exclusively on the AndroZoo training partition and subsequently evaluated directly on the complete CICMalDroid2020 and Drebin target datasets without any target-domain adaptation. Reported direct transfer scores represent the mean $\pm$ standard deviation across the five independently trained source models.

Scenario B: Limited Target-Domain Adaptation (10%)

To assess adaptability under limited supervision, the AndroZoo-trained source model was used as the initialization point for adaptation on each target dataset. Stratified repeated hold-out evaluation (five repetitions) was applied independently to CICMalDroid2020 and Drebin. In each repetition, 10% of the target dataset was randomly selected in a stratified manner for supervised adaptation, while the remaining 90% was reserved exclusively for evaluation. Reported adaptation scores represent the mean $\pm$ standard deviation across the five repetitions.

For RoBERTa, adaptation was performed through continued fine-tuning of the pre-trained source model. For Qwen3.5-27B, only LoRA adapter parameters were updated while the base model remained frozen, preserving source-domain knowledge while enabling lightweight domain adaptation. This protocol ensures strict separation between source-domain training and target-domain evaluation, preventing data leakage and ensuring that measured gains reflect either direct transferability or limited supervised adaptation rather than joint retraining.

4.5.3. Cross-Dataset Transfer Results

Table 16 presents the cross-dataset transfer performance of RoBERTa and Qwen3.5-27B+LoRA across three Android malware benchmarks. Both models demonstrate strong generalization capabilities, with Qwen3.5-27B+LoRA consistently outperforming RoBERTa across all settings. Under direct cross-domain transfer, RoBERTa maintains competitive performance with F1-scores of 0.912 (CICMal-Droid2020) and 0.908 (Drebin), representing only a modest decline from its in-domain AndroZoo performance (0.970). Notably, Qwen3.5-27B+LoRA exhibits superior transferability, achieving 0.954 and 0.952 F1-score on the same direct transfer tasks—a minimal degradation from its 0.983 in-domain result. With only 10% target-domain labeled data for adaptation, both architectures further improve: RoBERTa reaches 0.941 (CICMal) and 0.935 (Drebin), while Qwen3.5-27B+LoRA attains important results of 0.967 and 0.961, respectively. These findings indicate that (1) distributional shift between Android malware datasets, while present, can be effectively mitigated by representation learning models; (2) LLM-based encodings in Qwen3.5-27B+LoRA capture more transferable semantic features, yielding enhanced robustness to domain variation;

and (3) minimal target supervision (10%) enables efficient recalibration, pushing performance above the 0.95 F1 threshold across all benchmarks. The consistent outperformance of Qwen3.5-27B+LoRA (+0.03 to +0.04 F1 gain) underscores the value of semantically enriched pre-training for practical, drift-resilient malware detection.

Table 16. In-domain, direct cross-domain transfer, and limited target-domain adaptation performance (macro-averaged F1-score).

Dataset	AndroZoo (In-Domain)	CICMalDroid2020 (Direct Transfer)	CICMalDroid2020 (10% Target Adaptation)	Drebin (Direct Transfer)	Drebin (10% Target Adaptation)
RoBERTa	0.970 ± 0.003	0.912 ± 0.006	0.941 ± 0.006	0.908 ± 0.007	0.935 ± 0.006
Qwen3.5-27B+LoRA	0.982 ± 0.005	0.954 ± 0.007	0.967 ± 0.005	0.952 ± 0.005	0.961 ± 0.004

5. Threats to Validity

The proposed framework remains inherently constrained by its exclusive reliance on static analysis, which, although computationally efficient and highly scalable, cannot fully capture malicious behaviors that manifest only during runtime execution. Consequently, malware employing dynamic code loading, reflective invocation, environment-aware triggers, encrypted payload unpacking, or advanced anti-analysis mechanisms may evade detection, even when semantically enriched large language model representations are employed. Furthermore, while representation-learning architectures may offer improved robustness against superficial code manipulations compared with manually engineered feature pipelines, the present study does not include dedicated empirical evaluations under controlled obfuscation settings. Specifically, adversarial transformations such as string encryption, API renaming, junk code insertion, and control-flow flattening were not systematically assessed. Accordingly, the true resilience of semantic feature representations under standardized obfuscation scenarios remains an important open question for future investigation.

From a data-centric perspective, although external validation was conducted using CICMalDroid2020 and Drebin, the primary model development pipeline remains centered on the AndroZoo repository. While AndroZoo is widely recognized for its scale, accessibility, and metadata richness, its sampling characteristics and historical composition may not fully reflect the continuously evolving Android malware ecosystem or the latest behavioral trends in benign mobile applications. Although the cross-dataset transfer experiments presented in Section 4.5 demonstrate encouraging generalization capability, long-term operational robustness may still be affected by concept drift arising from evolving Android framework APIs, permission models, application architectures, and attacker tactics.

Additionally, despite implementing multiple safeguards to minimize experimental bias—including SHA-256 hash-level deduplication, strict APK-level separation between training and validation partitions, fold-specific preprocessing pipelines, and malware family distribution monitoring—guaranteeing complete independence between semantically related samples within large-scale public malware corpora remains methodologically difficult. Closely related variants originating from the same malware lineage may retain subtle structural or behavioral similarities that inadvertently influence representation learning and lead to mildly optimistic performance estimates. While cross-dataset experiments partially mitigate this concern by introducing independently curated benchmarks, residual family-level semantic overlap cannot be completely excluded.

The practical deployment feasibility of the proposed framework is further constrained by the computational demands of large-capacity language models. Although parameter-efficient adaptation through LoRA substantially reduces the overhead associated with full-model fine-tuning, architectures such as Qwen3.5-27B still require considerable GPU

memory and exhibit higher inference latency than lighter-weight alternatives. These constraints may limit direct deployment in resource-constrained operational environments, particularly in real-time mobile security monitoring scenarios.

Finally, the reported performance metrics were obtained under controlled experimental conditions using stratified cross-validation, repeated transfer evaluation, and curated benchmark datasets. While these evaluation settings provide strong comparative evidence, real-world deployment conditions often involve temporally evolving malware streams, incomplete labeling, non-stationary class distributions, and adversarial manipulation attempts. Consequently, additional validation under temporal split evaluation protocols, continuous concept drift monitoring, and adversarial robustness benchmarks would further strengthen confidence in the long-term operational reliability of the proposed framework.

Taken together, these limitations define the scope within which the present findings should be interpreted. At the same time, they identify clear future research directions, including hybrid static-dynamic analysis integration, adversarial robustness benchmarking, temporal generalization studies, and computational optimization of LLM-assisted Android malware detection pipelines.

6. Conclusions

This study presents a unified, reproducible framework for rigorously comparing three generations of Android malware detection paradigms: classical machine learning, Transformer-based architectures, and generative Large Language Models. By standardizing feature extraction, enforcing constraint-aware sequence selection, and introducing an LLM-driven feature distillation pipeline with parameter-efficient fine-tuning, this work addresses a methodological gap in the literature, where these paradigms have often been evaluated independently. Our end-to-end evaluation across 12,000 APKs demonstrates that each paradigm offers distinct advantages and that practical deployment requires balancing detection accuracy, computational efficiency, and interpretability.

Classical models, particularly Random Forest, establish a strong baseline (0.975 F1-score) with extremely low inference latency, confirming the continued relevance of well-engineered static feature pipelines for high-throughput screening tasks. Transformer architectures, led by RoBERTa (0.970 F1) and DistilBERT (0.951 F1), capture richer semantic dependencies while maintaining tractable inference times (~15–28 ms), making them suitable for scalable real-time analysis. The LLM-based framework achieved better performance among the evaluated models, with Qwen3.5-72B+LoRA reaching 0.982 ± 0.005 F1. However, this peak performance is strictly tied to the LoRA fine-tuning path and incurs significantly higher memory and latency costs compared to classical or distilled Transformer baselines, reinforcing the need for context-aware, tiered deployment rather than monolithic model replacement. Hallucination analysis further indicated a manageable error rate (7.7%), with limited downstream impact under controlled evaluation conditions, suggesting that LLM-assisted feature extraction can be practically usable within structured malware analysis workflows.

Beyond in-domain performance, cross-dataset transfer experiments on CICMal-Droid2020 and Drebin confirmed the substantial impact of distributional shift while also demonstrating that semantically enriched representations retain stronger adaptability under limited target-domain supervision. These findings suggest a practical tiered deployment strategy in which lightweight Transformer-based models support large-scale triage, while higher-capacity LLM-based models may be reserved for high-confidence secondary analysis and more explanation-oriented inspection tasks.

Several directions remain important for future work. Standardized adversarial benchmarking under controlled obfuscation pipelines is necessary to quantify robustness against

evasion techniques such as control-flow flattening, API renaming, and string encryption. Temporal validation under evolving malware ecosystems would further strengthen operational confidence under concept drift. Finally, multimodal fusion combining static semantic representations with dynamic execution traces, inter-component communication graphs, and network telemetry may help address the inherent limitations of static-only analysis.

Overall, the findings suggest that practical Android malware detection may benefit less from replacing one modeling paradigm with another, and more from strategically integrating complementary strengths across classical machine learning, Transformer architectures, and LLM-based semantic analysis.

Author Contributions: Conceptualization, E.T. and İ.A.D.; Methodology, E.T. and İ.A.D.; Validation, E.T.; Formal analysis, E.T.; Investigation, E.T. and İ.A.D.; Resources, E.T.; Data curation, E.T.; Writing—original draft, E.T. and İ.A.D.; Writing—review & editing, E.T. and İ.A.D.; Supervision, İ.A.D.; Project administration, İ.A.D.; Funding acquisition, E.T. and İ.A.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Gazi University /Scientific Research Projects Coordination Unit(BAP) grant number FDK-2022-7373.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets and code supporting this study are publicly available at <https://github.com/taskinegemen/article/>. This work utilizes the DREBIN (<https://drebin.mlsec.org/>, accessed on 3 October 2025), AndroZoo (<https://androzoo.uni.lu/>, accessed on 15 October 2025), and CICAndMal2020 (<https://www.unb.ca/cic/datasets/andmal2020.html>, accessed on 3 October 2025) datasets, accessible under their respective licensing terms. Additional details regarding dataset construction and preprocessing are available upon request for researchers.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

API	Application Programming Interface
APK	Android Package
AV	Antivirus
BERT	Bidirectional Encoder Representations from Transformers
BF16/FP8	BFloat16/Float8
C2	Command and Control
CFG	Control Flow Graph
CNN	Convolutional Neural Network
CSR	Compressed Sparse Row
DEX	Dalvik Executable
DL	Deep Learning
DT	Decision Tree
FPR	False Positive Rate
GB	Gradient Boosting
GPU	Graphics Processing Unit
IG	Information Gain
IMEI	International Mobile Equipment Identity
JSON	JavaScript Object Notation
KNN	k-Nearest Neighbors
LLM	Large Language Model
LoRA	Low-Rank Adaptation

LR	Logistic Regression
LSTM	Long Short-Term Memory
MI	Mutual Information
ML	Machine Learning
MLM	Masked Language Modeling
MLP	Multi-Layer Perceptron
NF4	NormalFloat 4-bit
NSP	Next Sentence Prediction
PEFT	Parameter-Efficient Fine-Tuning
RF	Random Forest
RNN	Recurrent Neural Network
RoBERTa	Robustly Optimized BERT Approach
SHA-256	Secure Hash Algorithm 256-bit
SVM	Support Vector Machine
TP/TN/FP/FN	True/False Positive/Negative
XML	Extensible Markup Language
XGBoost	Extreme Gradient Boosting

Appendix A

Table A1. Statistical comparison of advanced detection paradigms against the Random Forest baseline (macro-averaged F1-score).

Model	Mean F1 $\pm$ SD	Δ F1 vs. RF	Test	<i>p</i> -Value	Interpretation
Random Forest	0.975 $\pm$ 0.003	-	-	-	Baseline
RoBERTa (Multi-Level)	0.970 $\pm$ 0.003	−0.005	Wilcoxon	0.063	No significant difference
Qwen3.5-27B (Zero-Shot)	0.838 $\pm$ 0.012	−0.137	Paired <i>t</i> -test	<0.001	Significantly worse
Qwen3.5→RoBERTa	0.957 $\pm$ 0.005	−0.018	Paired <i>t</i> -test	0.002	Significantly worse
Qwen3.5-27B Fine-Tuned	0.982 $\pm$ 0.005	+0.007	Paired <i>t</i> -test	0.028	Higher mean performance, not significant after Bonferroni correction

Note: Wilcoxon signed-rank tests were applied when fold-wise performance differences violated the normality assumption; otherwise, paired two-tailed *t*-tests were used. Bonferroni-adjusted significance threshold: $\alpha_{adj} = 0.0125$, corresponding to four pairwise comparisons against the Random Forest baseline.

References

- Chakif, K.; Nawshin, F.; Unal, D. Robust Android Malware Detection against Obfuscation and Adversarial Attacks Using RGB Markov Images and Deep Ensemble Learning. *Knowl.-Based Syst.* **2026**, *337*, 115376. [\[CrossRef\]](#)
- Wu, Y.; Liu, Z.; Lin, Y.; Zhao, B.; Zhou, C.; Wu, T.; Hong, Z. Android App Repackaging Detection: A Comprehensive Survey. *Cyber Secur. Appl.* **2026**, *4*, 100124. [\[CrossRef\]](#)
- Maganur, S.; Jiang, Y.; Huang, J.; Zhong, F. Feature-Centric Approaches to Android Malware Analysis: A Survey. *Computers* **2025**, *14*, 482. [\[CrossRef\]](#)
- Wermke, D.; Huaman, N.; Acar, Y.; Reaves, B.; Traynor, P.; Fahl, S. A Large Scale Investigation of Obfuscation Use in Google Play. In *Proceedings of the 34th Annual Computer Security Applications Conference*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 222–235.
- Khalid, S.; Hussain, F.B.; Gohar, M. Towards Obfuscation Resilient Feature Design for Android Malware Detection-KTSODroid. *Electronics* **2022**, *11*, 4079. [\[CrossRef\]](#)
- Rafiq, H.; Aslam, N.; Aleem, M.; Issac, B.; Randhawa, R.H. AndroMalPack: Enhancing the ML-Based Malware Classification by Detection and Removal of Repacked Apps for Android Systems. *Sci. Rep.* **2022**, *12*, 19534. [\[CrossRef\]](#)
- Chen, Y.-C.; Chen, H.-Y.; Takahashi, T.; Sun, B.; Lin, T.-N. Impact of Code Deobfuscation and Feature Interaction in Android Malware Detection. *IEEE Access* **2021**, *9*, 123208–123219. [\[CrossRef\]](#)
- Yang, J.; Tang, J.; Yan, R.; Xiang, T. Android Malware Detection Method Based on Permission Complement and API Calls. *Chin. J. Electron.* **2022**, *31*, 773–785. [\[CrossRef\]](#)

9. Nganfang, V.; Queyruet, S.; Bromberg, D.; Schiavoni, V.; Mvondo, D.; Tchendji, V.K. DroidHunter: A Robust Vision-Based Detection Against Hidden Android Malware. In *Proceedings of the ASIACCS 2026—21st ACM ASIA Conference on Computer and Communications Security*, Bangalore, India, 1–5 June 2026.
10. Rathore, H.; Sahay, S.K.; Rajvanshi, R.; Sewak, M. Identification of Significant Permissions for Efficient Android Malware Detection. In *Proceedings of the Broadband Communications, Networks, and Systems*; Springer International Publishing: Cham, Switzerland, 2021.
11. Yerima, S.Y.; Sezer, S.; McWilliams, G.; Muttik, I. A New Android Malware Detection Approach Using Bayesian Classification. In *Proceedings of the 2013 IEEE 27th International Conference on Advanced Information Networking and Applications (AINA)*; IEEE: New York, NY, USA, 2013; pp. 121–128.
12. Arp, D.; Spreitzenbarth, M.; Hübner, M.; Gascon, H.; Rieck, K. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*; Internet Society: San Diego, CA, USA, 2014.
13. Jiang, X.; Mao, B.; Guan, J.; Huang, X. Android Malware Detection Using Fine-Grained Features. *Sci. Program.* **2020**, *2020*, 5190138. [\[CrossRef\]](#)
14. Ashawa, M.; Morris, S. Android Permission Classifier: A Deep Learning Algorithmic Framework Based on Protection and Threat Levels. *Secur. Priv.* **2021**, *4*, e164. [\[CrossRef\]](#)
15. Saleem, M.S.; Mišić, J.; Mišić, V.B. Android Malware Detection Using Feature Ranking of Permissions. *arXiv* **2022**. [\[CrossRef\]](#)
16. Brezinski, K.; Ferens, K. Metamorphic Malware and Obfuscation: A Survey of Techniques, Variants, and Generation Kits. *Secur. Commun. Netw.* **2023**, *2023*, 8227751. [\[CrossRef\]](#)
17. Andresini, G.; Appice, A.; Belvedere, V.; Fiameni, G.; Malerba, D. Anakin: Explainable Android Malware Detection with Graph Neural Networks. *Cybersecurity* **2026**, *9*, 116. [\[CrossRef\]](#)
18. Androguard. Core.Analysis Package—Androguard 3.4.0 Documentation. Available online: <https://androguard.readthedocs.io/en/latest/api/androguard.core.analysis.html> (accessed on 3 March 2026).
19. Yang, J.; Zhang, Z.; Zhang, H.; Fan, J. Android Malware Detection Method Based on Highly Distinguishable Static Features and DenseNet. *PLoS ONE* **2022**, *17*, e0276332. [\[CrossRef\]](#)
20. Muzaffar, A.; Hassen, H.R.; Zantout, H.; Lones, M.A. Reassessing Feature-Based Android Malware Detection in a Contemporary Context. *PLoS ONE* **2026**, *21*, e0341013. [\[CrossRef\]](#)
21. Kong, J.; Wang, G.; Chen, M.; Gu, W.; Zhang, Y.; Wu, Z. IFDroid: Enhancing Android Malware Detection Resilience Against Concept Drift Through API Sequence Intrinsic Features. *Int. J. Inf. Secur.* **2025**, *25*, 3. [\[CrossRef\]](#)
22. Kim, M.; Kim, D.; Hwang, C.; Cho, S.; Han, S.; Park, M. Machine-Learning-Based Android Malware Family Classification Using Built-In and Custom Permissions. *Appl. Sci.* **2021**, *11*, 10244. [\[CrossRef\]](#)
23. Mineau, J.-M.; Lalande, J.-F. Class Loaders in the Middle: Confusing Android Static Analyzers. *Digit. Threat.* **2025**, *6*, 1–19. [\[CrossRef\]](#)
24. Asmitha, K.A.; Vinod, P.; Rafidha Rehiman, K.A.; Raveendran, N.; Conti, M. Android Malware Defense through a Hybrid Multi-Modal Approach. *J. Netw. Comput. Appl.* **2025**, *233*, 104035. [\[CrossRef\]](#)
25. Feng, P.; Ma, J.; Li, T.; Ma, X.; Xi, N.; Lu, D. Android Malware Detection via Graph Representation Learning. *Mob. Inf. Syst.* **2021**, *2021*, 5538841. [\[CrossRef\]](#)
26. Guyton, F. Feature Selection on Permissions, Intents and APIs for Android Malware Detection. Ph.D. Thesis, Nova Southeastern University, Davie, FL, USA, 2021.
27. Kharnotia, S.; Arora, B.; Kour, R. Feature-Driven Static Analysis for Learning-Based Android Malware Detection: A Review. *ICT Express* **2026**, *12*, 186–208. [\[CrossRef\]](#)
28. Sihag, V.; Vardhan, M.; Singh, P. A Survey of Android Application and Malware Hardening. *Comput. Sci. Rev.* **2021**, *39*, 100365. [\[CrossRef\]](#)
29. Faruki, P.; Bhan, R.; Jain, V.; Bhatia, S.; El Madhoun, N.; Pamula, R. A Survey and Evaluation of Android-Based Malware Evasion Techniques and Detection Frameworks. *Information* **2023**, *14*, 374. [\[CrossRef\]](#)
30. Bhakta, D.; Abu Yousu, M.; Rana, M.S. Android Malware Detection Against String Encryption Based Obfuscation. In *Third Congress on Intelligent Systems*; Springer: Berlin/Heidelberg, Germany, 2002; pp. 543–555, ISBN 978-981-19-9378-7.
31. Ikram, M.; Beaume, P.; Kaafar, M.A. DaDiDroid: An Obfuscation Resilient Tool for Detecting Android Malware via Weighted Directed Call Graph Modelling. In *Proceedings of the 16th International Joint Conference on e-Business and Telecommunications (ICETE 2019)*; SciTePress: Setúbal, Portugal, 2019; pp. 211–219.
32. Hammad, M.; Garcia, J.; Malek, S. A Large-Scale Empirical Study on the Effects of Code Obfuscations on Android Apps and Anti-Malware Products. In *Proceedings of the 2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*; ACM: New York, NY, USA, 2018; pp. 421–431.
33. Calleja, A.; Martín, A.; Menéndez, H.D.; Tapiador, J.; Clark, D. Picking on the Family: Disrupting Android Malware Triage by Forcing Misclassification. *Expert Syst. Appl.* **2018**, *95*, 113–126. [\[CrossRef\]](#)

34. Anand, S.; Mitra, B.; Dey, S.; Rao, A.; Dhar, R.; Vaidya, J. MALITE: Lightweight Malware Detection and Classification for Constrained Devices. *IEEE Trans. Emerg. Top. Comput.* **2025**, *13*, 1099–1112. [\[CrossRef\]](#) [\[PubMed\]](#)
35. Qiu, J.; Zhang, J.; Luo, W.; Pan, L.; Nepal, S.; Xiang, Y. A Survey of Android Malware Detection with Deep Neural Models. *ACM Comput. Surv.* **2020**, *53*, 126. [\[CrossRef\]](#)
36. Kincl, J.; Eftimov, T.; Viktorin, A.; Šenkeřík, R.; Pavleska, T. Comprehensive Benchmarking of Knowledge Graph Embeddings Methods for Android Malware Detection. *Expert Syst. Appl.* **2025**, *288*, 127888. [\[CrossRef\]](#)
37. Khan, K.N.; Ullah, N.; Ali, S.; Khan, M.S.; Nauman, M.; Ghani, A. Op2Vec: An Opcode Embedding Technique and Dataset Design for End-to-End Detection of Android Malware. *Secur. Commun. Netw.* **2022**, *2022*, 3710968. [\[CrossRef\]](#)
38. Yuan, Z.; Lu, Y.; Wang, Z.; Xue, Y. Droid-Sec: Deep Learning in Android Malware Detection. In *Proceedings of the 2014 ACM Conference on SIGCOMM*; Association for Computing Machinery: New York, NY, USA, 2014; pp. 371–372.
39. Aldini, A.; Petrelli, T. Image-Based Detection and Classification of Android Malware through CNN Models. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 1–11.
40. Kunwar, P.; Aryal, K.; Gupta, M.; Abdelsalam, M.; Bertino, E. SoK: Leveraging Transformers for Malware Analysis. *IEEE Trans. Dependable Secur. Comput.* **2025**, *22*, 5888–5905. [\[CrossRef\]](#)
41. Alrawashdeh, K. Transformer-Based Memory Reverse Engineering for Malware Behavior Reconstruction. *Computers* **2025**, *15*, 8. [\[CrossRef\]](#)
42. Laskar, M.T.R.; Huang, J.X.; Hoque, E. Contextualized Embeddings Based Transformer Encoder for Sentence Similarity Modeling in Answer Selection Task. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*; Calzolari, N., Béchet, F., Blache, P., Choukri, K., Cieri, C., Declerck, T., Goggi, S., Isahara, H., Maegaard, B., Mariani, J., et al., Eds.; European Language Resources Association: Marseille, France, 2020; pp. 5505–5514.
43. Gaber, M.; Ahmed, M.; Janicke, H. Zero Day Malware Detection with Alpha: Fast DBI with Transformer Models for Real World Application. *Comput. Electr. Eng.* **2025**, *128*, 110751. [\[CrossRef\]](#)
44. Chaieb, M.; Ghorab, M.A.; Saied, M.A. Detecting Android Malware: From Neural Embeddings to Hands-On Validation with BERTroid. *arXiv* **2024**. [\[CrossRef\]](#)
45. Sabbah, A.; Jarrar, R.; Zein, S.; Mohaisen, D. Empirical Evaluation of Concept Drift in ML-Based Android Malware Detection. *arXiv* **2025**. [\[CrossRef\]](#)
46. Feng, R.; Chen, H.; Wang, S.; Monjurul Karim, M.; Jiang, Q. LLM-MalDetect: A Large Language Model-Based Method for Android Malware Detection. *IEEE Access* **2025**, *13*, 81347–81364. [\[CrossRef\]](#)
47. Priambodo, T.G.S.; Prabowo, A.O.; Puspitarini, A.D.; Winarso, R.A.H.; Aisyah, N.; Pratama, M.Y.; Purwitasari, D.; Pratomo, B.A. MalQwen: Fine Tuned LLM for Static Android Malware Analysis Report. *IEEE Access* **2025**, *13*, 208483–208497. [\[CrossRef\]](#)
48. Berger, H.; Hajaj, C.; Dvir, A. When the Guard Failed the Droid: A Case Study of Android Malware. *arXiv* **2020**. [\[CrossRef\]](#)
49. Veeresh, K.M.; Naik, B.R. Robust Android Malware Detection: Leveraging Generative Transformer-Based Feature Extraction and Hybrid Optimization Techniques. *SN Comput. Sci.* **2025**, *7*, 28. [\[CrossRef\]](#)
50. Gravereaux, S.C.; Islam, S.R. Accuracy and Efficiency Trade-Offs in LLM-Based Malware Detection and Explanation: A Comparative Study of Parameter Tuning vs. Full Fine-Tuning. *arXiv* **2025**. [\[CrossRef\]](#)
51. Bourebaa, F.; Benmohammed, M. Evaluating Lightweight Transformers with Local Explainability for Android Malware Detection. *IEEE Access* **2025**, *13*, 101005–101026. [\[CrossRef\]](#)
52. Trung, D.M.; Hao, T.D.A.; Minh, L.H.; Khoa, N.H.; Cam, N.T.; Pham, V.-H.; Duy, P.T. DMLDroid: Deep Multimodal Fusion Framework for Android Malware Detection with Resilience to Code Obfuscation and Adversarial Perturbations. *arXiv* **2025**. [\[CrossRef\]](#)
53. Gu, J.; Zhu, H.; Han, Z.; Li, X.; Zhao, J. GSEDroid: GNN-Based Android Malware Detection Framework Using Lightweight Semantic Embedding. *Comput. Secur.* **2024**, *140*, 103807. [\[CrossRef\]](#)
54. Naseer, M.; Ullah, F.; Ijaz, S.; Naeem, H.; Alsirhani, A.; Alwakid, G.N.; Alomari, A. Obfuscated Malware Detection and Classification in Network Traffic Leveraging Hybrid Large Language Models and Synthetic Data. *Sensors* **2025**, *25*, 202. [\[CrossRef\]](#) [\[PubMed\]](#)
55. Hou, R.; Tian, X.; Geng, G. A Malware Detection Method Based on LLM to Mine Semantics of API. *EAI Endorsed Trans. AI Robot.* **2025**, *4*, 1–14. [\[CrossRef\]](#)
56. Jelodar, H.; Meymani, M.; Razavi-Far, R.; Ghorbani, A.A. XGen-Q: An Explainable Domain-Adaptive LLM Framework with Retrieval-Augmented Generation for Software Security. *arXiv* **2025**. [\[CrossRef\]](#)
57. Soi, D.; Sanna, A.; Maiorca, D.; Giacinto, G. Enhancing Android Malware Detection Explainability through Function Call Graph APIs. *J. Inf. Secur. Appl.* **2024**, *80*, 103691. [\[CrossRef\]](#)
58. Zhao, W.; Wu, J.; Meng, Z. AppPoet: Large Language Model Based Android Malware Detection via Multi-View Prompt Engineering. *Expert Syst. Appl.* **2025**, *262*, 125546. [\[CrossRef\]](#)

59. Qian, X.; Zheng, X.; He, Y.; Yang, S.; Cavallaro, L. LAMD: Context-Driven Android Malware Detection and Classification with LLMs. *arXiv* **2025**. [CrossRef]
60. Lan, T.; Naït-Abdesselam, F. LLM-Driven Feature-Level Adversarial Attacks on Android Malware Detectors. *arXiv* **2025**. [CrossRef]
61. Zheng, X.; Qian, X.; He, Y.; Yang, S.; Cavallaro, L. Beyond Classification: Evaluating LLMs for Fine-Grained Automatic Malware Behavior Auditing. *arXiv* **2025**. [CrossRef]
62. Yang, L.; Zheng, T.; Chen, Y.; Xiu, K.; Zhou, H.; Wang, D.; Zhao, P.; Qin, Z.; Ren, K. HarmMetric Eval: Benchmarking Metrics and Judges for LLM Harmfulness Assessment. *arXiv* **2026**. [CrossRef]
63. He, Y.; She, H.; Qian, X.; Zheng, X.; Chen, Z.; Qin, Z.; Cavallaro, L. On Benchmarking Code LLMs for Android Malware Analysis. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*; ACM: New York, NY, USA, 2025; pp. 153–160.
64. De Sutter, B.; Schrittwieser, S.; Coppens, B.; Kochberger, P. Evaluation Methodologies in Software Protection Research. *ACM Comput. Surv.* **2024**, *57*, 86. [CrossRef]
65. Hu, X.; Fu, Z.; Xie, S.; Ding, S.H.H.; Charland, P. SoK: Potentials and Challenges of Large Language Models for Reverse Engineering. *arXiv* **2025**. [CrossRef]
66. Liu, G.; Caragea, D.; Ou, X.; Roy, S. Benchmarking Android Malware Detection: Rethinking the Role of Traditional and Deep Learning Models. *arXiv* **2025**. [CrossRef]
67. Alkahtani, H.; Aldhyani, T.H.H. Artificial Intelligence Algorithms for Malware Detection in Android-Operated Mobile Devices. *Sensors* **2022**, *22*, 2268. [CrossRef]
68. Su, M.-Y.; Fung, K.-T. Detection of Android Malware by Static Analysis on Permissions and Sensitive Functions. In *Proceedings of the 2016 Eighth International Conference on Ubiquitous and Future Networks (ICUFN)*; IEEE: New York, NY, USA, 2016; pp. 873–875.
69. Jusoh, R.; Firdaus, A.; Anwar, S.; Osman, M.Z.; Darmawan, M.F.; Ab Razak, M.F. Malware Detection Using Static Analysis in Android: A Review of FeCO (Features, Classification, and Obfuscation). *PeerJ Comput. Sci.* **2021**, *7*, e522. [CrossRef] [PubMed]
70. Jerome, Q.; Allix, K.; State, R.; Engel, T. Using Opcode-Sequences to Detect Malicious Android Applications. In *Proceedings of the 2014 IEEE International Conference on Communications (ICC)*; IEEE: New York, NY, USA, 2014; pp. 914–919.
71. Arslan, R.S. JDroid: Android Malware Detection Using Hybrid Opcode Feature Vector. *PeerJ Comput. Sci.* **2025**, *11*, e3051. [CrossRef] [PubMed]
72. Mousavi, Z.; Islam, C.; Babar, M.A.; Abuadbba, A.; Moore, K. Detecting Misuse of Security APIs: A Systematic Review. *ACM Comput. Surv.* **2025**, *57*, 303. [CrossRef]
73. Li, L.; Bissyandé, T.F.; Outeau, D.; Klein, J. DroidRA: Taming Reflection to Support Whole-Program Analysis of Android Apps. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 318–329.
74. Wang, W.; Gao, Z.; Zhao, M.; Li, Y.; Liu, J.; Zhang, X. DroidEnsemble: Detecting Android Malicious Applications with Ensemble of String and Structural Static Features. *IEEE Access* **2018**, *6*, 31798–31807. [CrossRef]
75. Ebad, S.A.; Darem, A.A. Exploring Android Obfuscators and Deobfuscators: An Empirical Investigation. *Electronics* **2024**, *13*, 2272. [CrossRef]
76. El-Zawawy, M.A.; Faruki, P.; Conti, M. Formal Model for Inter-Component Communication and Its Security in Android. *Computing* **2022**, *104*, 1839–1865. [CrossRef]
77. Compatibility Framework Changes (Android 12). Available online: <https://developer.android.com/about/versions/12/reference/compat-framework-changes> (accessed on 21 April 2026).
78. El-Zawawy, M.A.; Hamdy, A. Detection of Hidden Privilege Escalations in Android. *Autom. Softw. Eng.* **2025**, *32*, 68. [CrossRef]
79. Mariconti, E.; Onwuzurike, L.; Andriotis, P.; De Cristofaro, E.; Ross, G.; Stringhini, G. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models. In *Proceedings of the Network and Distributed System Security Symposium*; Internet Society: San Diego, CA, USA, 2017.
80. Abedin, M.M.-H.-Z.; Mehruh, T. Comparison of Multiple Classifiers for Android Malware Detection with Emphasis on Feature Insights Using CICMalDroid 2020 Dataset. In *Proceedings of the 2025 IEEE 7th International Conference on Sustainable Technologies for Industry 5.0 (STI)*; IEEE: New York, NY, USA, 2025; pp. 1–6.
81. Jung, J.; Park, J.; Cho, S.; Han, S.; Park, M.; Cho, H.-H. Feature Engineering and Evaluation for Android Malware Detection Scheme. *J. Internet Technol.* **2021**, *22*, 423–439. [CrossRef]
82. Kouliaridis, V.; Potha, N.; Kambourakis, G. Improving Android Malware Detection Through Dimensionality Reduction Techniques. In *Proceedings of the Machine Learning for Networking: Third International Conference, MLN 2020, Paris, France, 24–26 November 2020*; Revised Selected Papers; Springer: Berlin/Heidelberg, Germany, 2020; pp. 57–72.
83. Arif, J.M.; Razak, M.F.A.; Awang, S.; Mat, S.R.T.; Ismail, N.S.N.; Firdaus, A. A Static Analysis Approach for Android Permission-Based Malware Detection Systems. *PLoS ONE* **2021**, *16*, e0257968. [CrossRef]

84. Arun, N.; Nisha Dayana, T.R. Machine Learning Based Multilayered Feature Analysis for Android Malware Detection. In *Proceedings of the 2025 6th International Conference on Electronics and Sustainable Communication Systems (ICESC)*; IEEE: New York, NY, USA, 2025; pp. 1558–1564.
85. Rahali, A.; Akhloufi, M.A. MalBERT: Using Transformers for Cybersecurity and Malicious Software Detection. *arXiv* **2021**. [[CrossRef](#)]
86. Souani, B.; Khanfir, A.; Bartel, A.; Allix, K.; Le Traon, Y. Android Malware Detection Using BERT. In *Proceedings of the Applied Cryptography and Network Security Workshops*; Zhou, J., Adepu, S., Alcaraz, C., Batina, L., Casalicchio, E., Chattopadhyay, S., Jin, C., Lin, J., Losiouk, E., Majumdar, S., et al., Eds.; Springer International Publishing: Cham, Switzerland, 2022; pp. 575–591.
87. Khan, I.; Kwon, Y.-W. A Structural-Semantic Approach Integrating Graph-Based and Large Language Models Representation to Detect Android Malware. In *Proceedings of the ICT Systems Security and Privacy Protection*; Pitropakis, N., Katsikas, S., Furnell, S., Markantonakis, K., Eds.; Springer Nature: Cham, Switzerland, 2024; pp. 279–293.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.